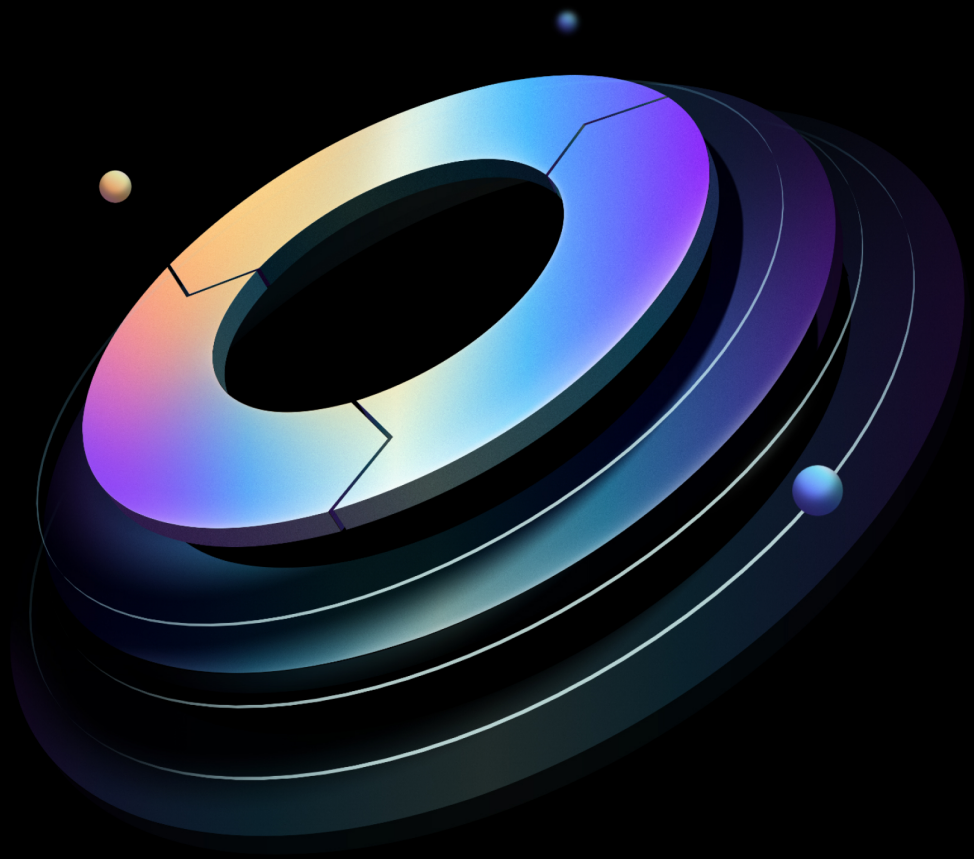
HashiCorp

Consul Installation Guide

Built from commit 3a6a26c83c

HashiCorp | Validated Designs

09 November 2026



HashiCorp

Contents

1 Introduction **6**
1.1 Why use HashiCorp Validated Designs? . . . 6
1.2 Objective . . . 6
1.3 Audience . . . 7
1.4 Supported versions . . . 7
1.5 Language and definitions . . . 7

2 Prerequisites and personnel **9**
2.0.1 Roles and responsibilities . . . 9
2.1 Multi-cloud and multi-runtime preparation . . . 11
2.1.1 People . . . 11
2.1.2 Infrastructure creation using Terraform . . . 12
2.1.3 Consul installation via Helm on Kubernetes . . . 13
2.1.4 Setting up admin partitions on-premises . . . 13
2.1.5 Setting up cluster peering between multiple clusters . . . 13

3 What is Consul? **13**
3.1 Consul Concepts . . . 13
3.1.1 Service discovery . . . 13
3.1.2 Service mesh . . . 16
3.2 Core components . . . 17
3.2.1 Control plane . . . 17
3.2.2 Dataplane . . . 18

4 Enterprise features **20**
4.0.1 Availability and performance . . . 21
4.0.2 Scalability . . . 21
4.0.3 Operational excellence . . . 21
4.0.4 Security . . . 21

5 Architecture - Consul on VM **22**
5.1 Recommended deployment architecture . . . 22
5.2 Consul agent configuration . . . 24
5.2.1 Consul server example configuration . . . 24

5.2.2 Consul client example configuration . . . 26

6 Architecture - Consul on Kubernetes 28
6.1 Consul on Kubernetes . . . 28
6.2 Kubernetes deployment architecture . . . 28
6.3 Component compatibility . . . 32

7 Sizing guidelines 32
7.1 Sizing guidelines . . . 32
7.1.1 Control plane on VMs . . . 33
7.2 Control plane on Kubernetes . . . 35

8 Detailed design 35
8.1 Roles and responsibilities . . . 35
8.1.1 Personnel . . . 36
8.1.2 Access . . . 37
8.2 Shared services . . . 37
8.2.1 DNS . . . 37
8.2.2 Security . . . 39
8.2.3 Secrets . . . 47
8.2.4 Monitoring . . . 49
8.2.5 Metrics . . . 49
8.2.6 Logs . . . 51
8.3 VM-based technical requirements . . . 51
8.3.1 Infrastructure . . . 53
8.4 Kubernetes Based control plane technical requirements . . . 59
8.4.1 Infrastructure . . . 61
8.4.2 Compute . . . 61
8.4.3 Networking . . . 62

9 Deploying Consul using Terraform 69
9.1 Platform-specific guidance . . . 69
9.2 Deployment sequence overview . . . 70
9.3 Preparation . . . 71
9.3.1 Create the TLS certificate files . . . 71
9.3.2 Obtain the Consul Enterprise license file . . . 71

9.3.3 Download and install the Consul CLI . . . 72
9.3.4 Download and install the Terraform CLI . . . 72
9.3.5 Configure your cloud credentials. . . . 72
9.4 Installation . . . 73
9.4.1 Set up your prerequisite resources . . . 73
9.4.2 Initialize your Terraform workspace . . . 75
9.4.3 Configure variables for the deployment . . . 75
9.4.4 Create and apply the Terraform plan . . . 76
9.4.5 Verify that the cluster has been successfully created and is accessible . . . 78

10 Multi-cloud and multi-runtime deployment 79
10.1 Consul preparation . . . 79
10.1.1 Deploy infrastructure . . . 79
10.1.2 Prerequisites . . . 79
10.1.3 Consul networking configuration . . . 79
10.1.4 Consul multi-tenancy configuration . . . 79
10.2 Consul installation and deployment . . . 80
10.2.1 Install Consul on Kubernetes . . . 80
10.2.2 Deploy Consul on EC2 . . . 82
10.2.3 Setup Consul and Nomad . . . 82
10.3 Consul networking . . . 85
10.3.1 Mesh gateways deployed over VMs . . . 85
10.3.2 Mesh gateways deployed on Kubernetes . . . 85
10.3.3 Mesh gateways in Nomad . . . 86
10.3.4 Setup DNS . . . 88
10.4 API Gateway Configuration . . . 88
10.4.1 API Gateway in Kubernetes . . . 88
10.4.2 API gateway in Nomad . . . 89
10.4.3 Deploy the API gateway . . . 90
10.5 Setup cluster peering . . . 92
10.5.1 Establishing cluster peering connection diagram . . . 92
10.5.2 Enable cluster peering in Kubernetes . . . 92
10.5.3 Enable cluster peering for VMs . . . 92
10.5.4 Sameness groups . . . 93
10.5.5 Service failover . . . 94

10.6 References 95

11 Post-installation configuration 95
11.1 Configure DNS forwarding 95
11.1.1 Local agent resolution with DNS forwarding 96
11.1.2 External resolution via DNS forwarding 97
11.1.3 Kubernetes DNS forwarding with CoreDNS 97
11.1.4 Forwarding destination for external resolution and kubernetes configurations 98
11.1.5 DNS ACL permissions 98
11.1.6 DNS performance considerations 98

12 Identity integration — initial setup 100
12.1 Configure initial ACL roles and policies 101
12.1.1 Token precedence 101
12.1.2 Bootstrap the acl system 102
12.1.3 Configure user authentication 103
12.1.4 Configure machine authentication 107
12.1.5 Service identities 109
12.1.6 Agent tokens and node identities 109
12.1.7 AWS EC2 authentication 110
12.1.8 Configuration as code authentication 110

13 Observability setup 110
13.0.1 Monitoring 110
13.0.2 Metrics 111
13.0.3 Logs 113

14 Backup and restore — initial setup 113

15 Disaster recovery — initial setup 114

16 Next steps 118
16.1 Operational handoff 118
16.2 Consumer onboarding 118
16.3 Legacy summary folded into next steps 119

17 Appendix **119**
17.1 Appendix 1: Admin partitions . . . 119
17.2 Appendix 2: Kubernetes catalog sync . . . 120
17.3 Appendix 3: Gossip protocol . . . 120
17.4 Appendix 4: Sample Terraform and helm configuration . . . 121
17.4.1 Consul server . . . 121
17.4.2 Consul client . . . 122
17.4.3 Helm configuration - load balancer . . . 124
17.4.4 ELB_IRSA - Terraform configuration . . . 125
17.4.5 AWS load balancer controller - TF . . . 125
17.4.6 EBS CSI driver . . . 126
17.4.7 Consul server helm . . . 128
17.4.8 Non-default partition K8s cluster - helm . . . 130
17.4.9 Agent telemetry . . . 131
17.4.10 Consul telemetry collector intention . . . 131
17.4.11 Consul server sizing - EC2 to Consul cluster size . . . 132
17.4.12 Agents . . . 132

18 Changelog **134**
18.1 2026-08 . . . 134
18.2 2026-01 . . . 134
18.3 2025-06 . . . 134
18.4 2025-05 . . . 134
18.5 2025-01 . . . 134
18.6 2024-02 . . . 134

19 Credits **135**

1 Introduction

> **Note**
>
> These guides were reorganized in August 2026. The previous Solution Design Guide and Operating Guides (Adoption, Standardization, Scaling) have been replaced with three role-based guide types: Installation, Administration, and User. Read the HVD 2.0 release notes for full details on what changed and where to find the content you need.

1.1 Why use HashiCorp Validated Designs?

HashiCorp Validated Designs (HVD) provide practitioners with opinionated guidance for achieving production-grade deployments of HashiCorp products. These designs are purpose-built for delivering foundational use cases, with a baseline level of architectural and operational maturity. They draw on the field experiences of Solutions Engineers and Solutions Architects working with customers across a wide range of environments and organizational requirements.

Each guide provides access to an opinionated reference architecture, including key design decisions and the rationale behind them. Where applicable, guides identify modular design components that you can adjust to align with organizational or regulatory requirements without compromising the overall integrity of the implementation. For many deployments we include Terraform modules to automate large portions of infrastructure provisioning and software installation.

1.2 Objective

The objective of this Consul Installation Guide is to assist you in deploying Consul Enterprise. Consul allows users to register services for service discovery use cases in a centralized registry, enabling the discovery, monitoring, and tracking of service health. Additionally, it supports service mesh use cases for secure service-to-service communication across multiple environments, platforms, and runtimes.

This guide addresses these objectives using two architectural patterns:

- Consul on virtual machines (VMs)
- Consul on Kubernetes

These patterns are adaptable to various cloud providers and on-premises environments, ensuring flexibility in deployment.

> **Note**
>
> The recommendations in this Installation Guide are derived from different production Consul deployments. Before you implement these recommendations, carefully evaluate them and determine if it is acceptable for your specific environment.

1.3 Audience

This document is designed for development, platform, networking, and application security teams seeking to implement Consul Enterprise to register, discover, and securely connect applications across cloud and datacenter environments.

This Installation Guide is for platform, infrastructure, security, and operations practitioners deploying self-hosted Consul Enterprise on virtual machines and Kubernetes. By the end of this guide, the Consul control plane is deployed, initially configured, and ready for day-2 administration and consumer use-case onboarding.

After deployment, use the Consul Administration Guide for day-2 operations and the Consul User Guide for service discovery, service mesh, and application-team workflows.

1.4 Supported versions

This version of the guide has been validated with the following versions of Consul Enterprise:

- Consul Enterprise 1.19.x+ent

1.5 Language and definitions

This documentation uses technology-agnostic terminology; however, some terms may vary slightly between cloud providers. Below are the definitions of key terms used throughout this document.

Cloud provider terms	Definition
Region	A physical location with multiple clusters of data centers globally.
Availability zone (AZ)	One or more distinct data centers within a region, each with redundant power, networking, and connectivity.
Public subnet	A network accessible to application users.
Private subnet	A network used internally by applications, not accessible to application users.
Secrets Manager (SM)	A system used to securely store and manage secrets for bootstrapping and configuration.
Virtual private cloud (VPC)	A software-defined cloud networking environment.

Consul terms	Definition
Datacenter	The smallest unit of Consul infrastructure capable of performing basic operations. It may align with cloud region boundaries.
Partition or Admin Partition	A logical boundary within a single Consul datacenter that separates distinct network boundaries or teams.
Peering or Cluster Peering	A Consul datacenter or admin partition with an established relationship with a remote datacenter or partition.

Attribute	Definition
Availability	Design features that minimize the impact of subsystem failures on overall solution uptime.
Operational excellence	Best practices that ensure efficient management and operation of the system.
Performance	How design decisions affect the overall performance of the solution.
Scalability	The ability of design decisions to allow the service usage and control plane to handle increasing workloads.
Security	How a design decision affects the security posture of the overall solution.

2 Prerequisites and personnel

This section identifies the roles, access, and prerequisite infrastructure required before deploying Consul Enterprise.

2.0.1 Roles and responsibilities

Requirements and recommendations for operating a secure Consul deployment may vary drastically depending on your intended workloads, operating system, and environment. This section will provide architecture recommendations to meet the technical requirements of your organization and deploy Consul to meet the requirements. This section defines the roles required from the participants, and includes authority and access needs to the working environment.

Personnel

Focusing on the deployment activity alone, we recommend enlisting a project leader and a Cloud administration team. A project leader primarily coordinates events, facilitates resources and assigns duties to the Cloud administration team.

Members of the Cloud administration team carry out functional tasks to install Consul Enterprise. For Cloud administrators, we make assumptions about general knowledge of the following:

- Cloud architecture and administration
- Administration-level experience with Linux
- Peripheral awareness of Docker
- Practical knowledge of Terraform

In addition, we strongly recommend requesting assistance from a Security Operations team. The emphasis is on integrating formal security controls required for services hosted in your preferred cloud environment.

Finally, it is essential to specify a role for the infrastructure automation team who will take over the deployment at the end of this exercise.

Persona Title	Role Description	Responsibilities
Platform/Infrastructure Engineer	Define & implement the infrastructure decisions.	Manages Consul Service discovery and Service Mesh platform.
Security Engineer/Architect	Define and implement security framework in the platform	Manage security policies, access control, secrets and authorization policies.
Developer	Write codes and maintain applications.	Implements service-to-service communications and maintains deployment definitions.

Access

The installation team requires administrative access to various resources to install and configure Consul Enterprise including:

- **Compute/storage**: compute instances, Kubernetes clusters, object storage
- **Network**: VPC, firewalls, load balancer, etc.

- **TLS certificates**: Load balancer security
- **Identity**: AWS IAM, AAD, Cloud IAM, etc.
- **Secrets management and data encryption key management**: AWS Secrets Manager, Azure KeyVault, Cloud Key Management, etc.

2.1 Multi-cloud and multi-runtime preparation

This section describes how platform teams will need to prepare the infrastructure for a large-scale deployment spanning on-premises and multi-cloud environments. The platform team will be responsible for infrastructure deployment, maintaining DevOps tooling, monitoring, and maintaining several clouds.

The following workflow outlines the key steps for deploying Consul across various environments, including Kubernetes, virtual machines (VMs), and Nomad. Each stage ensures that the necessary prerequisites are installed, Consul is deployed and configured, applications are launched, and communication between environments is established through Consul peering connections.

2.1.1 People

Platform team

The platform team manages the deployment of VM-based, Nomad, and Kubernetes infrastructure. They should be responsible for the following actions:

- Infrastructure creation via Terraform.
- Consul installation via Terraform for VMs.
- Consul installation via Helm for Kubernetes using GitOps.
- Setting up admin partitions on-premises.
- Setting up cluster peering between on-premises, AWS, and Azure deployments.

Development teams

Development teams deploy applications. On-premise, they should be responsible for installing a dashboard application on VMs and installing a counting application in Nomad. For cloud environments, they should look at installing a dashboard app on AWS and a counter app in Azure.

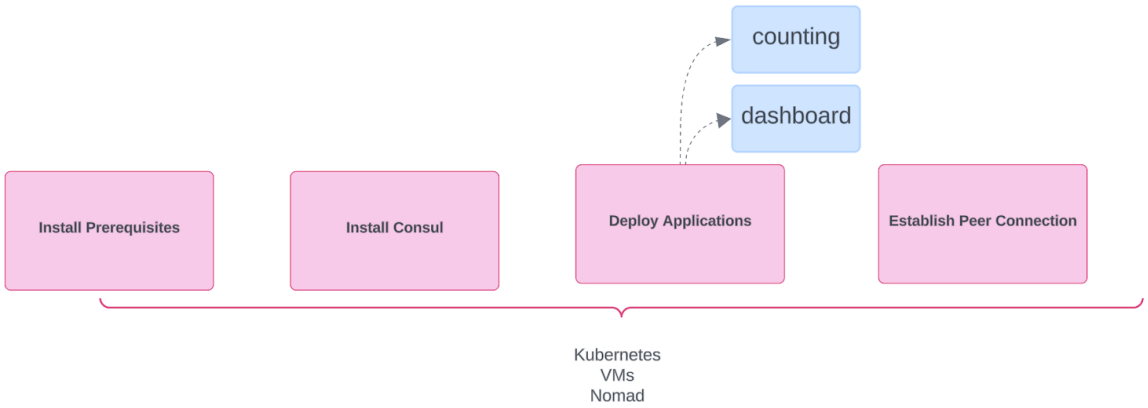


Figure 1: Initial config diagram

2.1.2 Infrastructure creation using Terraform

On-premise Consul install on VMs

Platform teams can follow the recommended deployment architecture discussed in the Consul Installation Guide to right-size the deployment, check component compatibility, and follow the deployment sequence using the provided Terraform modules.

On-premise Nomad install

Nomad and Consul client installation steps are provided in the deployment section below. For reference, the Nomad HVD provides initial configuration guidance.

Infrastructure creation for Kubernetes on AWS and Azure

The Terraform Installation Guide provides platform-specific guidance for creation. Teams can choose the Kubernetes distribution they prefer. This documentation focuses on Elastic Kubernetes Service (EKS) and Azure Kubernetes Service (AKS) specifically.

2.1.3 Consul installation via Helm on Kubernetes

Once the infrastructure is built, Consul can be installed on the Kubernetes cluster. HashiCorp publishes a Helm chart for Consul. The Helm values that will be used are discussed later on in the document.

2.1.4 Setting up admin partitions on-premises

Admin partitions are discussed in detail in the Administration Guide. In this guide, you will set up admin partitions in the on-premises cloud. This is so separate frontend and backend teams can have full administrative control of their respective partitions. Each of these teams will operate autonomously within their partition without the burden of managing the Consul control plane.

2.1.5 Setting up cluster peering between multiple clusters

Cluster peering is also introduced in the Administration Guide. In this guide, you will connect on-premises and Kubernetes clusters running on AWS and Azure using cluster peering. Cluster peering is also discussed in detail in the following sections.

3 What is Consul?

HashiCorp Consul is a service networking solution that helps teams manage and secure network connectivity between services across on-premises, multi-cloud environments, and various runtimes. Consul provides features such as service discovery, service mesh, traffic management, and automated updates to network infrastructure devices. These features can be used individually or integrated within a single Consul deployment.

3.1 Consul Concepts

3.1.1 Service discovery

A popular use case for Consul is its service discovery capabilities. Service instances are registered to a central registry that monitors them through health checks. Utilizing a service identity, rather

than traditionally an IP address and port, the service catalog can dynamically track and report service instance changes within a service catalog. Consumers then utilize DNS, HTTP APIs and Consul's CLI to dynamically retrieve other service's information from the service catalog.

Consul provides a control plane that enables you to register, query, and secure services deployed across your network. The control plane is the part of the network infrastructure that maintains a central registry to track services and their respective IP addresses.

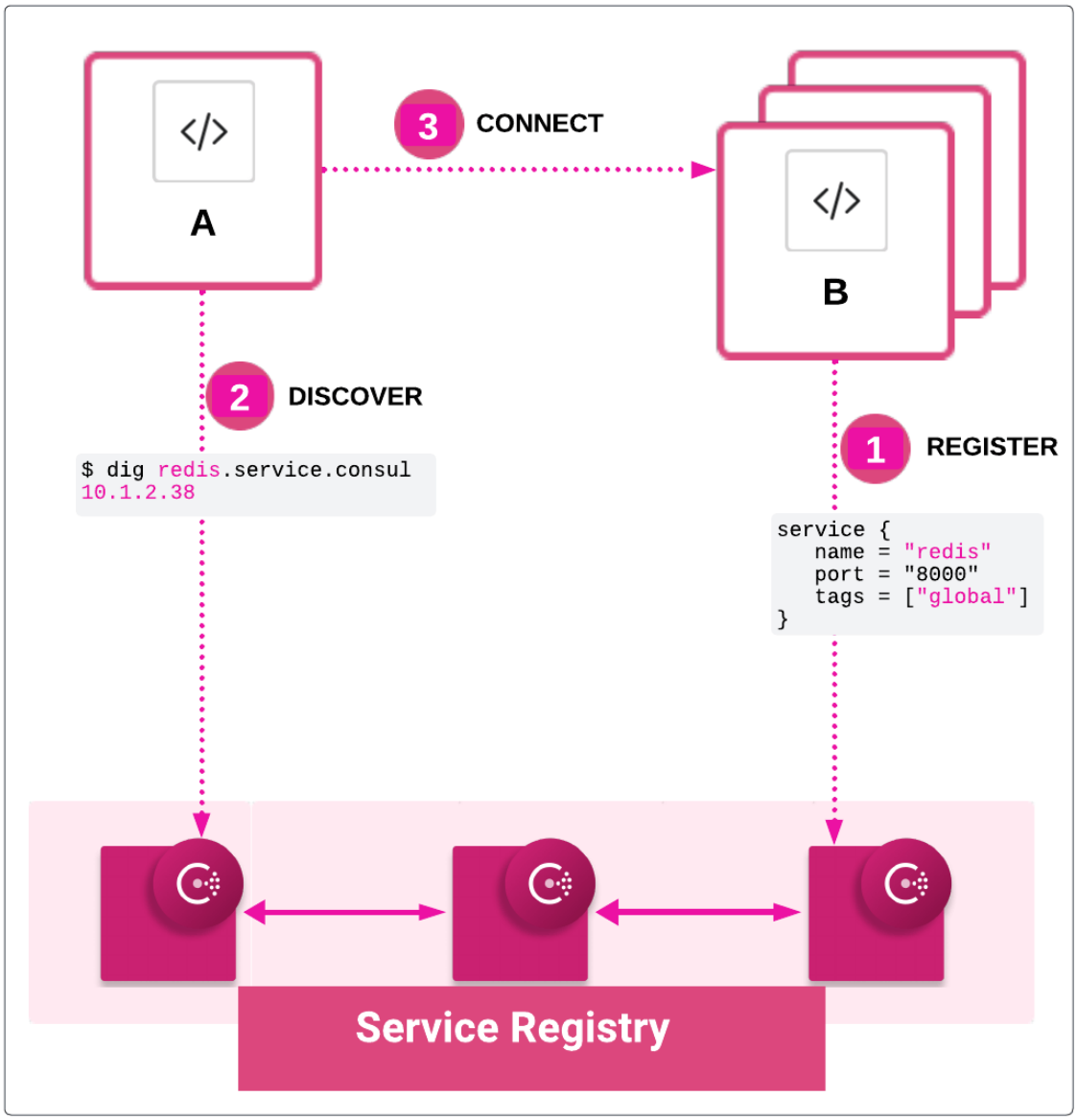


Figure 2: Consul Service Registry

3.1.2 Service mesh

Consul's service mesh provides secure service-to-service connection, authorization, and encryption within the mesh. Leveraging mutual transport layer security (mTLS), sidecar proxies run in the mesh to ensure service communication security without requiring changes to application logic. Consul service mesh allows you to divide application layer traffic between different subsets of service instances. You can leverage L7 traffic management capabilities to perform complex processes, such as configuring backup services for failover scenarios, canary and A-B testing, blue-green deployments. In addition, service mesh intentions enable secure access control. Intentions control traffic between services at both the networking layer (Layer 4) and the application layer (Layer 7).

When using Consul's service mesh capabilities, Consul interacts with the data plane, the part of the network infrastructure that processes data requests. Consul dynamically configures sidecar and gateway proxies in the request path, which enables you to authorize service-to-service connections, route requests to healthy service instances, and enforce mTLS encryption without modifying your service's code. This ensures that communication remains performant and reliable.

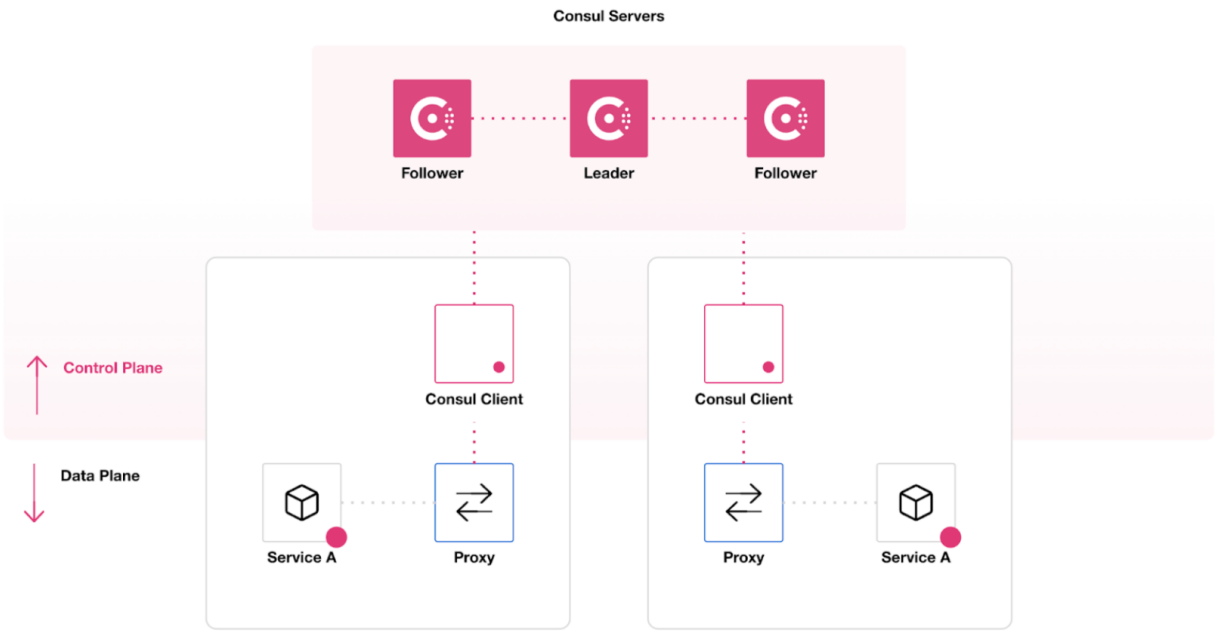


Figure 3: Consul Logical Architecture

3.2 Core components

3.2.1 Control plane

The **Control plane** is responsible for the duties pertaining to health, management and configuration of Consul agents (server, clients) and consul-dataplane. This includes, but is not limited to:

- Cluster state and replication
- Cluster membership between Consul Server and clients.
- Managing data plane proxy configurations:
  - Application proxies
  - Consul Gateway proxies.
- Health checks for Consul Servers, Consul clients, applications and application proxies.
- Cluster peering relationships and which services are shared between peered clusters.

Consul server

Consul servers store all the configuration data for your infrastructure. The Consul server maintains a catalog of each service, with its IP address and port number, allowing services to discover and communicate with one another. The server also provides APIs for managing nodes, services, health checks, and more.

We recommend deploying 6 servers in a cluster. The more servers you deploy, the greater the resilience and availability in the event of a failure. More servers, however, slow down consensus, which is a critical server function that enables Consul to efficiently and effectively process information.

Consul server clusters elect a single server to be the leader through consensus. The leader processes all queries and transactions, which prevents conflicting updates in clusters containing multiple servers.

Servers that are not currently acting as the cluster leader are called followers. Followers forward write requests from client agents to the cluster leader. The leader replicates the write requests to all other servers in the cluster. Replication ensures that if the leader is unavailable, other servers in the cluster can elect another leader without losing any data. Read requests can be serviced by followers depending on the specified consistency mode.

When using redundancy zones, you can configure non-voting members to improve Consul resiliency and to enhance read scalability. If voting followers become unavailable, Consul would

attempt to keep server quorum by promoting a non-voting member to become a voting follower. In normal operations, non-voting members enhance read scalability by increasing the number of servers that can service a read request.

In standard deployments, Consul uses a control plane that contains both server agents and client agents. Server agents maintain the service catalog and service mesh, including its security and consistency, while client agents manage communications between service instances, their sidecar proxies, and the servers. This model is optimal for applications deployed on virtual machines or bare metal servers.

You must use this model if you have services running on VMs.

Control Plane for Kubernetes (consul-k8s)

The `consul-k8s-control-plane` binary includes first-class integrations between Consul and Kubernetes. The project encapsulates multiple use cases such as syncing services, injecting Consul sidecars, and more.

Following features are covered in this Installation Guide:

- **Catalog Sync**: Sync Consul services into first-class Kubernetes services and vice versa. This enables Kubernetes to easily access external services and for non-Kubernetes nodes to easily discover and access Kubernetes services.
- **Consul Service Mesh**: Run Consul Service Mesh on Kubernetes. This feature injects Envoy sidecars and registers your Pods with Consul.

3.2.2 Dataplane

Consul clients

For standard VM, bare metal, and Nomad workloads, Consul uses an agent-based model that relies on client agents. You must run client agents on every compute node in your datacenter. Consul clients accept service registrations and report node and service health status to the Consul cluster.

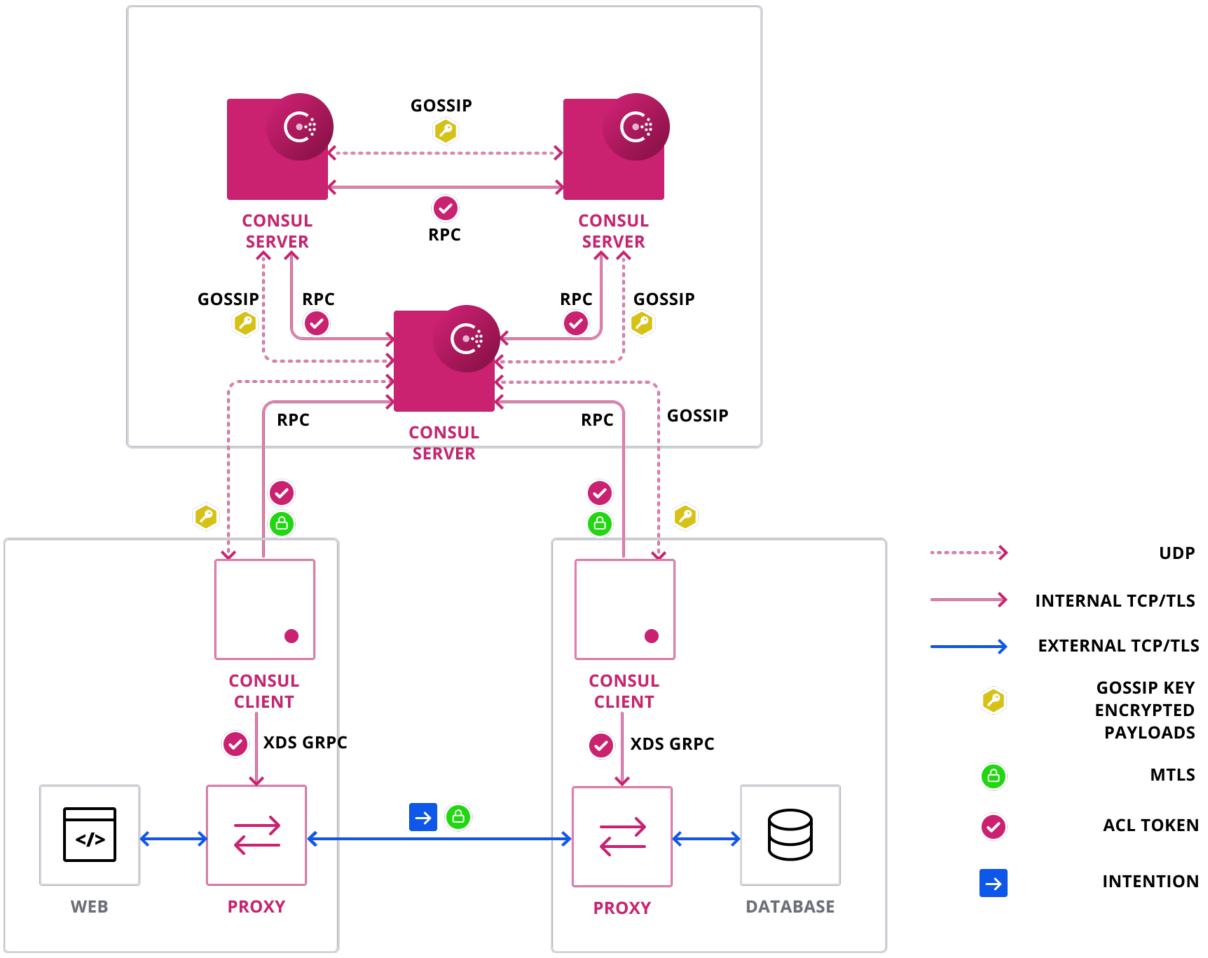


Figure 4: Consul Clients

Consul-dataplane on kubernetes

Consul on Kubernetes (as of Consul v1.14) relies on consul-dataplane (referenced as Dataplane in the diagram below) which is a lightweight process for managing Envoy proxies. Consul Dataplane removes the need to run client agents on every node in a cluster for service discovery and service mesh. Instead, Consul deploys sidecar proxies that provide lower latency, support additional runtimes, and integrate with cloud infrastructure providers.

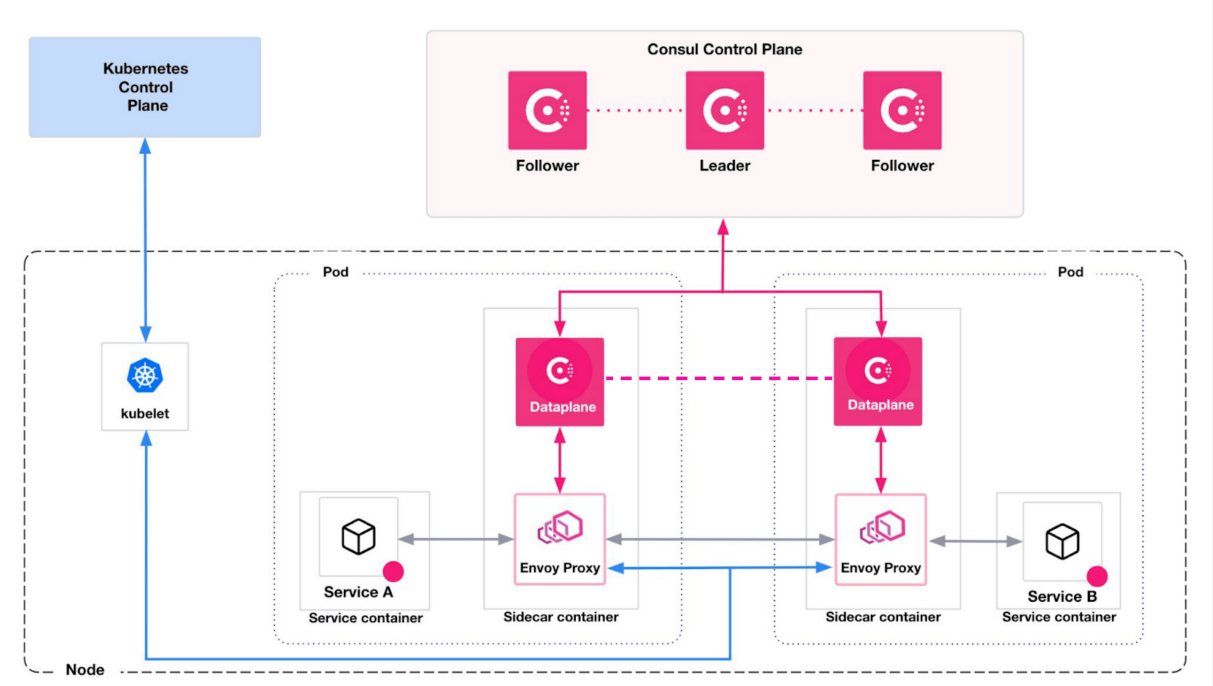


Figure 5: Consul Dataplane

4 Enterprise features

This Hashicorp Validated Design is prescriptive technical content for HashiCorp Enterprise products and solutions.

The following features are available in Consul Enterprise and will be part of our recommendations.

> Note
>
> Consul Enterprise features address the organizational complexity of operating, scaling and governing Consul.

4.0.1 Availability and performance

- **Sameness groups:** Define partitions and cluster peers as a member of a group to streamline configuration or easily configure failover to members of the group.
- **Redundancy zones:** Deploy backup voting Consul servers to efficiently improve the Consul fault tolerance.
- **Server request rate limit per source IP:** Limit gRPC and RPC traffic to servers by source IP addresses.

4.0.2 Scalability

- **Read replicas:** Deploy non-voting Consul servers to enhance the scalability of the read requests.

4.0.3 Operational excellence

- **Autopilot/automated upgrades:** Assist in upgrades of the Consul cluster via version updates.
- **Automated backups:** Configure and assist in the backup of the cluster state.
- **Admin partitions:** Define administrative boundaries between tenants within a single consul datacenter.
- **Namespaces:** Define resource boundaries within a single admin partition for further organizational flexibility.

4.0.4 Security

- **OIDC auth method:** Manage user access to Consul through an OIDC identity provider instead of Consul ACLs directly.
- **Audit logging:** Capture Consul access and usage patterns by reviewing access to Consul HTTP API.
- **FIPS 140-2 compliance:** Leverage FIPS builds of Consul Enterprise to ensure Consul deployments are secured with BoringCrypto and CNGCrypto, and compliant with FIPS 140-2.

5 Architecture - Consul on VM

5.1 Recommended deployment architecture

We recommend deploying the control plane with 6 server nodes in the Consul cluster, evenly distributed across three availability zones. Configure Consul Autopilot to enable the redundancy zones feature, mapping each availability zone to a Consul redundancy zone. Autopilot will maintain 1 voting server per zone, while the remaining server in each zone will act as a non-voting server. These non-voting servers improve read scalability and can be promoted to voting servers in the event of a voting server failure.

Note

This section outlines the architecture components and best practices for deploying Consul on virtual machines in a production environment. By leveraging the cloud provider's security and high availability features alongside the recommended architecture, you can ensure the availability, scalability, and security of both the Consul control plane and data plane.

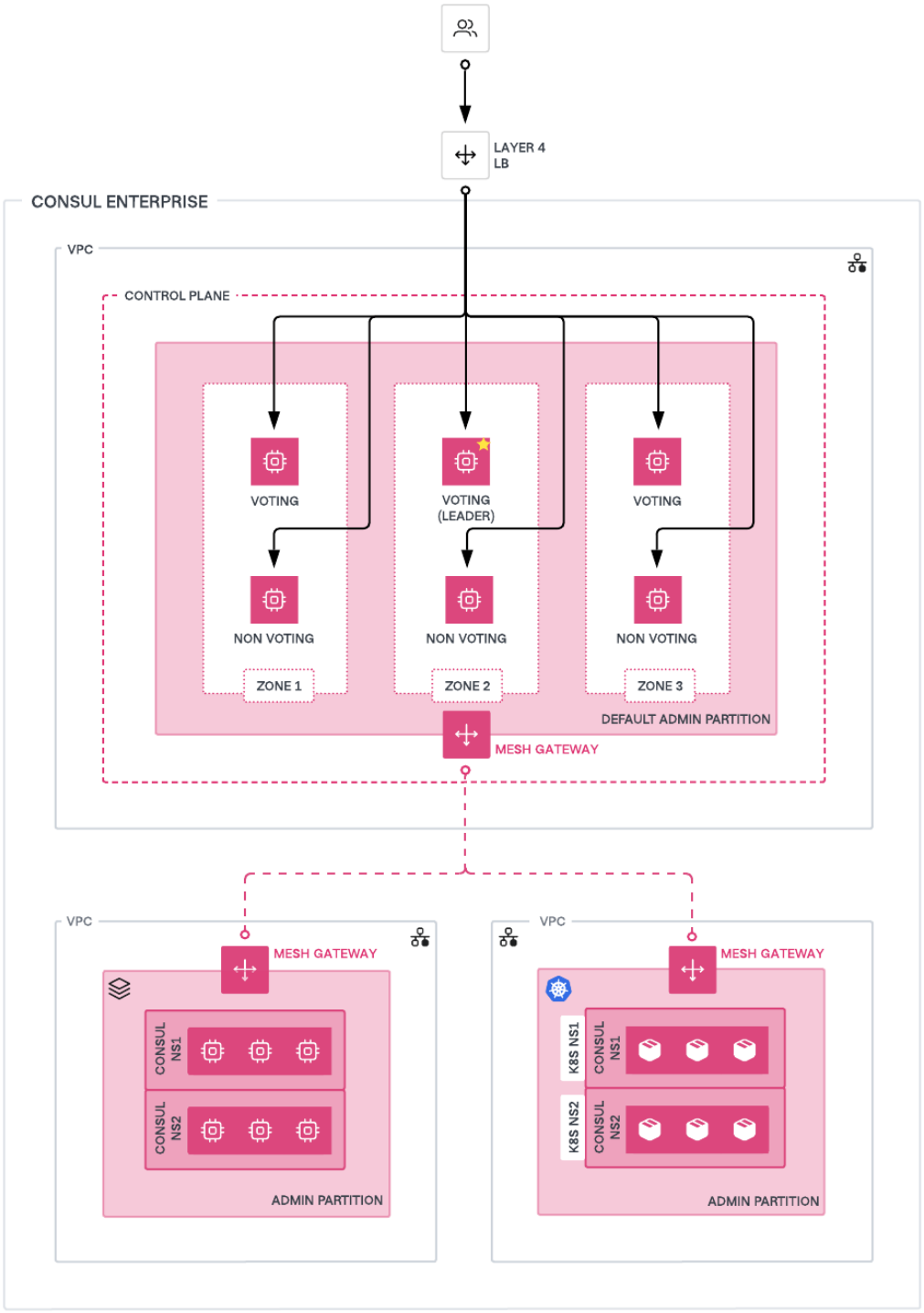


Figure 6: consul architecture

5.2 Consul agent configuration

You can create one or more configuration files to configure the Consul agent. The configuration files are formatted as HCL or JSON. The Installation Guide recommends HCL and will deploy server configuration in HCL.

The below configuration is required to start the Consul agents in server (`server.hcl`) and client (`client.hcl`) mode respectively.

5.2.1 Consul server example configuration

The following is an example Consul server agent configuration.

server.hcl

AWS

```
data_dir = "/opt/consul"
log_level = "${consul_log_level}"

datacenter = "${consul_datacenter}"

encrypt = "${gossip_encryption_key}"
encrypt_verify_incoming = true
encrypt_verify_outgoing = true

server = true
license_path = "${consul_license_path}"

# Configure Redundancy Zones
autopilot {
  redundancy_zone_tag = "zone"
}
node_meta {
  zone = "${consul_server_availability_zone}"
}

# Enable ACLs
acl {
  enabled = true
  default_policy = "deny"
}

client_addr = "0.0.0.0"

ports = {
  "grpc" = -1
  "grpc_tls" = 8503
  "http" = -1
  "https" = 8501
}

enable_central_service_config = true

advertise_addr = "$INTERNAL_IP_ADDRESS"

retry_join = ["provider=aws tag_key=Environment-Name tag_value=primary-consul"
    ]

# TLS config
tls {
  defaults {
    verify_incoming = false
    verify_outgoing = true
    ca_file = "/etc/consul.d/tls/consul-ca.pem"
    ca_path = "/etc/consul.d/tls"
    cert_file = "/etc/consul.d/tls/consul-cert.pem"
    key_file = "/etc/consul.d/tls/consul-key.pem"
  }

# overrides tls.defaults path, if specified.
```

5.2.2 Consul client example configuration

The following is an example Consul client agent configuration.

client.hcl

AWS

```
data_dir = "/opt/consul"
node_name = "consul-client-`hostname`"
log_level = "${consul_log_level}"
datacenter = "${consul_datacenter}"

encrypt = "${gossip_encryption_key}"
server = false

client_addr = "0.0.0.0"

partition = "${admin_partition}"

ports = {
  "grpc" = -1
  "grpc_tls" = 8503
  "http" = -1
  "https" = 8501
}

advertise_addr = "$INTERNAL_IP_ADDRESS"
retry_join = ["provider=aws tag_key=Environment-Name tag_value=primary-consul"
    ]

# TLS config
tls {
 defaults {
    verify_incoming = false
    verify_outgoing = true
    ca_file = "/etc/consul.d/tls/consul-ca.pem"
    ca_path = "/etc/consul.d/tls"
    cert_file = "/etc/consul.d/tls/consul-cert.pem"
    key_file = "/etc/consul.d/tls/consul-key.pem"
  }

  # overrides tls.defaults path, if specified.
  https {
    verify_incoming = true
    verify_outgoing = true
  }
  grpc {
    verify_incoming = false
  }
  internal_rpc {
    verify_server_hostname = true
  }
}
auto_encrypt {
  allow_tls = true
}

#Telemetry
telemetry {
  prometheus_retention_time = "480h",
  disable_hostname = true
}
```

6 Architecture - Consul on Kubernetes

6.1 Consul on Kubernetes

Preparing your Kubernetes environment to successfully deploy and run Consul is an important first step in your production deployment process. This solution will help you prepare your AWS Elastic Kubernetes Service (EKS). Since Consul servers are responsible for maintaining the datacenter state, server sizing is critical for overall performance, efficiency, and health of the Consul datacenter.

There are many possible configurations for running Consul on Kubernetes.

1. Consul servers deployed outside of Kubernetes and clients (catalog sync/consul-dataplane) inside kubernetes (**this is the recommended pattern**).
2. Consul servers (control plane) and clients (catalog sync/consul-dataplane) can be entirely within Kubernetes (**not recommended**).

6.2 Kubernetes deployment architecture

As a pre-requisite, ensure consul servers are up and running on VM as per recommendation above.

The below `values.yaml` file shows how to configure the Helm chart to install Consul so that it joins that existing Consul server cluster.

```
global:
  enabled: false

externalServers:
  enabled: true
  hosts: [<consul server DNS, IP or exec= string>]
  k8sAuthMethodHost: 'https://kubernetes.example.com:443'
```

Service discovery with Consul on Kubernetes requires Catalog Sync, refer to Service discovery catalog for detailed implementation of Catalog Sync. Here is the architecture diagram for this use-case.

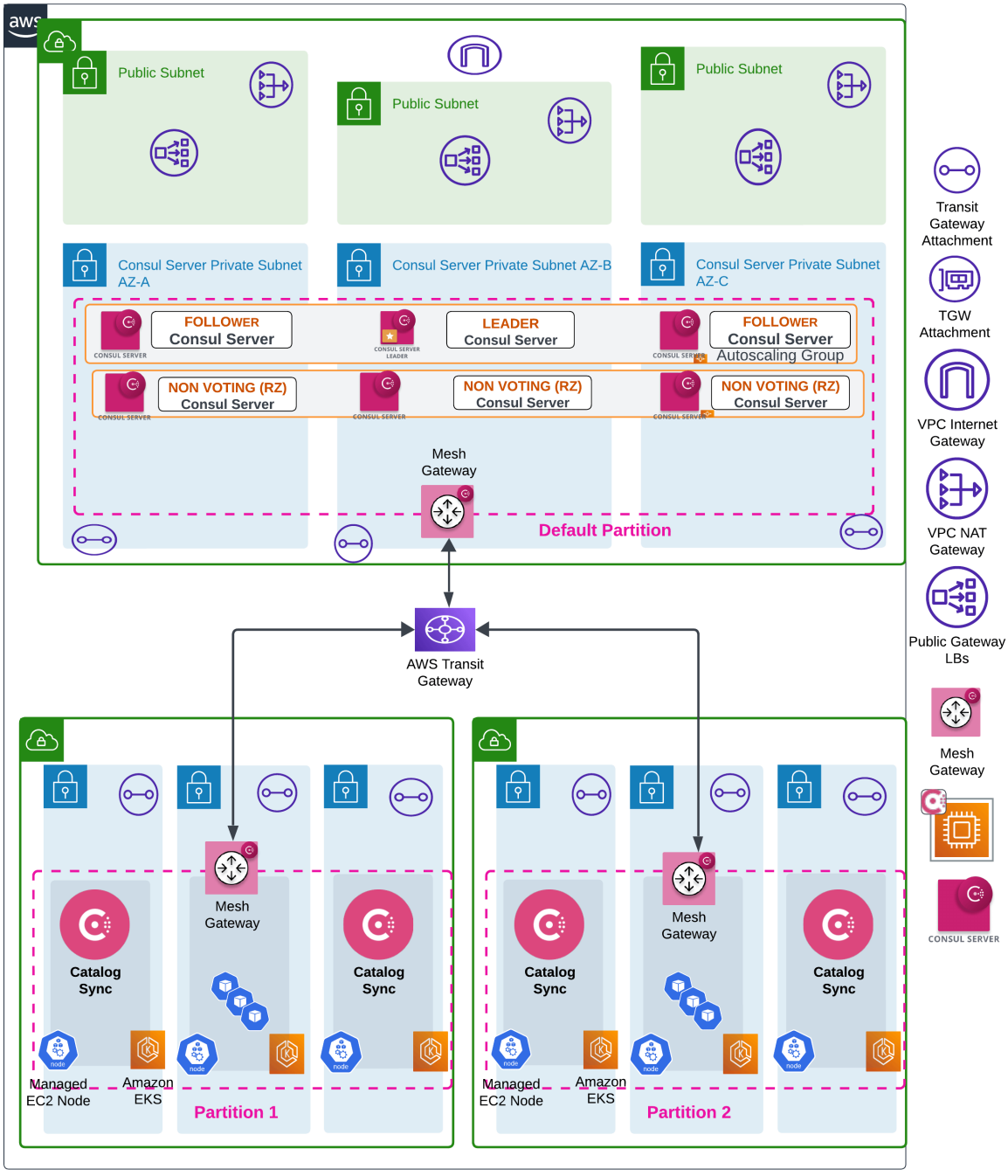


Figure 7: consul EKS architecture

For service mesh use-case, with the introduction of Consul Dataplane, Consul installation on Kubernetes is simplified by removing the Consul Client agents. This requires the Helm installation and rest of the consul-k8s components installed on Kubernetes to talk to Consul Servers directly on various ports. Before starting the installation, ensure that the Consul Servers are configured to have the gRPC port enabled `8502/tcp` using the `ports.grpc = 8502` configuration option. Here is the architecture diagram for this use-case.

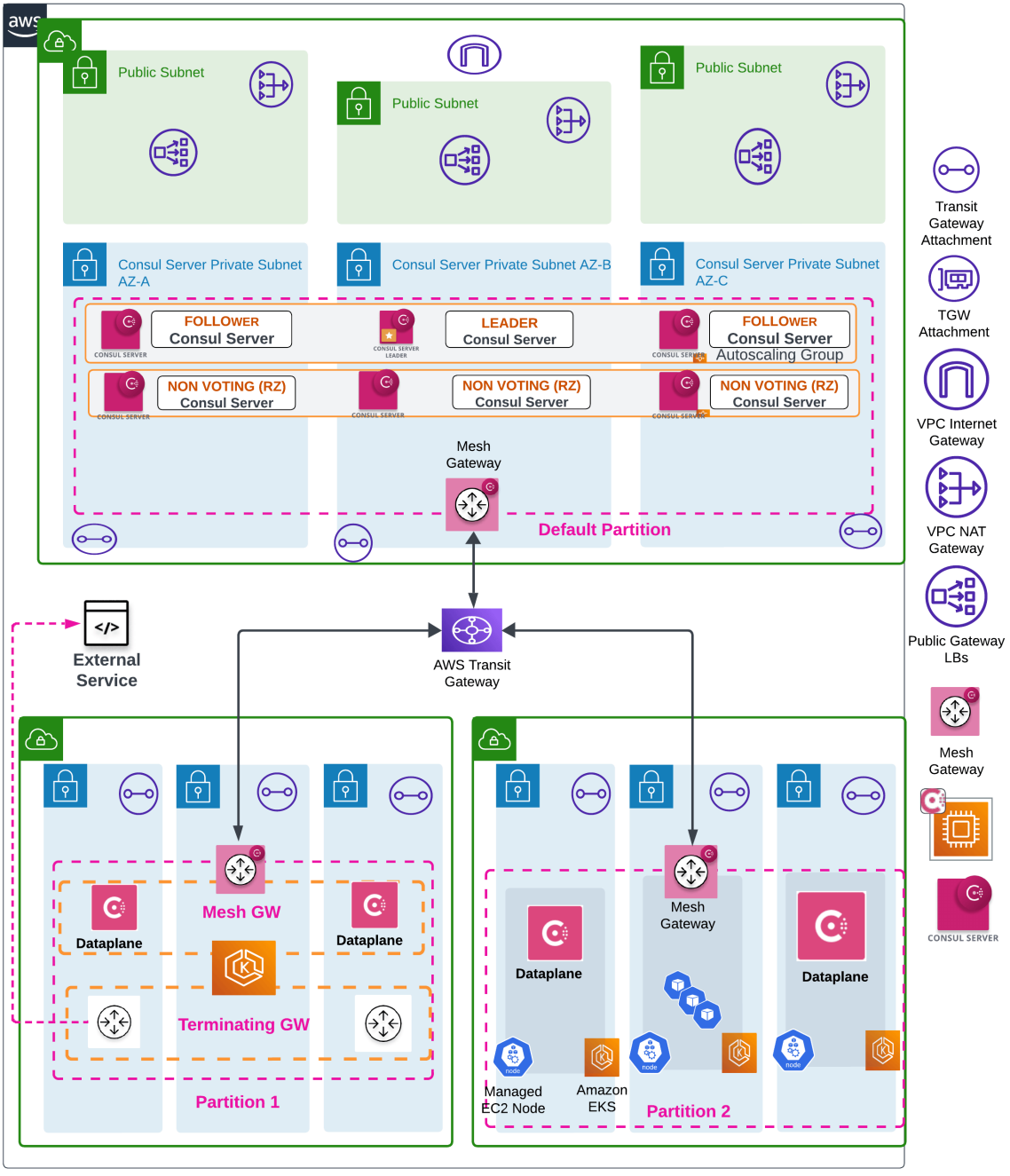


Figure 8: consul EKS architecture

6.3 Component compatibility

Component Name	Supported Implementation	Technologies and Versions
Platform	Kubernetes	AWS Elastic Kubernetes Service (EKS); Managed Node group; 1.27
Consul Enterprise Version	Linux Binary	Latest Consul Enterprise version - consul_1.16.X+ent
Object Storage	S3 compatible	AWS S3 - Consul Snapshot backups
Key Management	KMS	AWS KMS - Initial secrets and certs storage
Load Balancer	NLB or ALB	AWS Network Load Balance (NLB) - Layer 4 LB; AWS Application Load Balancer (ALB) - Layer 7 LB
Secret Manager	AWS SSM	AWS Secrets Manager

7 Sizing guidelines

7.1 Sizing guidelines

The Consul server nodes process all read and write operations from the agents as well as maintain a consensus among the cluster, and as such are I/O bound for writes and CPU bound for reads. This needs to be taken into consideration and monitoring put in place to adjust as required depending on the type of workload inside of the cluster.

Workloads on virtual machines require Consul client agents for service discovery and service mesh. So the following guidelines are based on that requirement.

As a general rule, we recommend that the maximum size for a single datacenter is 5,000 Consul client agents. This estimate is based on impact of recovery time, write and read requests, and other

factors. We recommend deploying read replicas for improved scalability in clusters that are read-heavy. We have customers who have scaled Consul to tens of thousands of agents per cluster, but it is highly dependent on the read and write workloads of the cluster. As such, customers must optimize for stability at the gossip layer as the cluster scales. The two main factors that affect this with client agent are:

1. Total size of the gossip pool
2. The churn of nodes/agents in the pool

7.1.1 Control plane on VMs

We recommend deploying at a minimum the following types of instances for the Consul Servers. These are broken down into Initial and Large clusters. We recommend starting with the Initial cluster size and once adoption occurs, vertically scaling the servers to the Production Cluster size.

AWS

Size	Potential Instance Type	CPU	Memory	Disk Capacity	Disk IO
Initial	m5.large	2	8	min: 100 GB (gp3)	min: 3000 IOPS
Small	m5.xlarge	4	16	min: 100 GB (gp3)	min: 3000 IOPS
Large	m5.2xlarge	8	32	min: 200 GB (gp3)	min: 7500 IOPS
Extra-Large	m5.4xlarge	16	64	min: 200 GB (gp3)	min: 7500 IOPS

GCP

Size	Potential Instance Type	CPU	Memory	Disk Capacity	Disk IO
Initial	n2-standard-2	2	8	min: 100 GB (pd-balanced)	min: 3000 IOPS
Small	n2-standard-4	4	16	min: 100 GB (pd-balanced)	min: 3000 IOPS
Large	n2-standard-8	8	32	min: 200 GB (pd-ssd)	min: 7500 IOPS
Extra-Large	n2-standard-16	16	64	min: 200 GB (pd-ssd)	min: 7500 IOPS

Azure

Size	Potential Instance Type	CPU	Memory	Disk Capacity	Disk IO
Initial	Standard_D2s	2	8	min: 100	min: 3000

The above architecture will support a high level of agents based clients, but we highly recommend that if a single datacenter in the above architecture is provisioned, that customers monitor cluster metrics to both establish a baseline and set threshold levels.

7.2 Control plane on Kubernetes

Use the CPU and memory recommendations to set resource limits for the Consul pods, and apply the disk recommendations when configuring persistent volumes. Both limits and requests should be set in the Helm chart. Below is an example Helm configuration snippet for deploying a Consul server in a large environment.

```
server:
  resources: |
    requests:
      memory: "32Gi"
      cpu: "8"
    limits:
      memory: "32Gi"
      cpu: "8"

  storage: 200Gi
```

HashiCorp recommends monitoring your production deployment to take data-driven informed decisions to scale your production server resource limits or vertically scale the VM deployments.

8 Detailed design

8.1 Roles and responsibilities

Requirements and recommendations for operating a secure Consul deployment may vary drastically depending on your intended workloads, operating system, and environment. This section will provide architecture recommendations to meet the technical requirements of your organization and deploy Consul to meet the requirements. This section defines the roles required from the participants, and includes authority and access needs to the working environment.

8.1.1 Personnel

Focusing on the deployment activity alone, we recommend enlisting a project leader and a Cloud administration team. A project leader primarily coordinates events, facilitates resources and assigns duties to the Cloud administration team.

Members of the Cloud administration team carry out functional tasks to install Consul Enterprise. For Cloud administrators, we make assumptions about general knowledge of the following:

- Cloud architecture and administration
- Administration-level experience with Linux
- Peripheral awareness of Docker
- Practical knowledge of Terraform

In addition, we strongly recommend requesting assistance from a Security Operations team. The emphasis is on integrating formal security controls required for services hosted in your preferred cloud environment.

Finally, it is essential to specify a role for the infrastructure automation team who will take over the deployment at the end of this exercise.

Persona Title	Role Description	Responsibilities
Platform/Infrastructure Engineer	Define & implement the infrastructure decisions.	Manages Consul Service discovery and Service Mesh platform.
Security Engineer/Architect	Define and implement security framework in the platform	Manage security policies, access control, secrets and authorization policies.
Developer	Write codes and maintain applications.	Implements service-to-service communications and maintains deployment definitions.

8.1.2 Access

The installation team requires administrative access to various resources to install and configure Consul Enterprise including:

- **Compute/storage**: compute instances, Kubernetes clusters, object storage
- **Network**: VPC, firewalls, load balancer, etc.
- **TLS certificates**: Load balancer security
- **Identity**: AWS IAM, AAD, Cloud IAM, etc.
- **Secrets management and data encryption key management**: AWS Secrets Manager, Azure KeyVault, Cloud Key Management, etc.

8.2 Shared services

The detailed design section covers the Control plane deployment architecture on AWS EC2 VM and on AWS EKS (Kubernetes). This segregates the architecture design in two separate sections. We have shared underlying infrastructure that can span across Control plane architecture and multi-platform client deployments on AWS EC2 VM's or on the AWS EKS (Kubernetes).

Shared services design decision covers the element that spans across multi-platform Consul deployment and allows users to correctly configure security groups (firewall rules) correctly to allow Consul DNS work properly, configure consul security which spans across Consul Admin partitions or Consul cluster peers. This section also covers Consul monitoring to implement the correct telemetry for the required use-cases.

8.2.1 DNS

For Consul Clients to be able to register services to the Consul Cluster, as well as utilize the cluster for Service Discovery and DNS requests, DNS Forwarding will need to be implemented within AWS.

There are multiple ways to forward DNS; BIND, iptables, and dnsmasq to name a few from a system level. However, it is recommended to implement DNS forwarding at a higher level, and we have two recommendations for AWS customers.

The first recommendation, for customers who are leveraging Cloud Native architectures, and utilizing Amazon DNS, is to establish an outbound Route53 resolver to point to the DNS service of

Consul (by default, port 8600). The Amazon DNS resolver is the .2 address of the VPC CIDR block. All DNS requests will be sent to this .2 address for resolution and if needed forwarded onto other Name Servers for query resolution. A Route53 resolver rule to forward all Consul resolution requests should be configured on the Route53 resolver to allow agents and other clients to send queries to the Consul Control plane.

The Route53 Resolver Rule will be associated with a Route53 Outbound Resolver Endpoint, and this rule can also be applied to multiple VPCs. You may want to have a Route53 Outbound Endpoint local to every VPC with individual rules for management. However, we recommend centralizing the Endpoints in the Consul VPC. This will allow clients to utilize the endpoints in different VPCs within the same AWS Account that the Consul Control Plane is deployed into, as well as allow you to share the Route53 Resolver rule with additional accounts in your AWS Organization with AWS Resource Access Manager (RAM). By sharing the Resolver rule, as well as having reachability to the Consul Control Plane VPC (via Transit Gateway, VPN, VPC-peering, etc), all VPCs can send traffic to the Route53 Outbound Resolver Endpoints to forward onto the Consul domain. An example of this traffic flow is shown below:

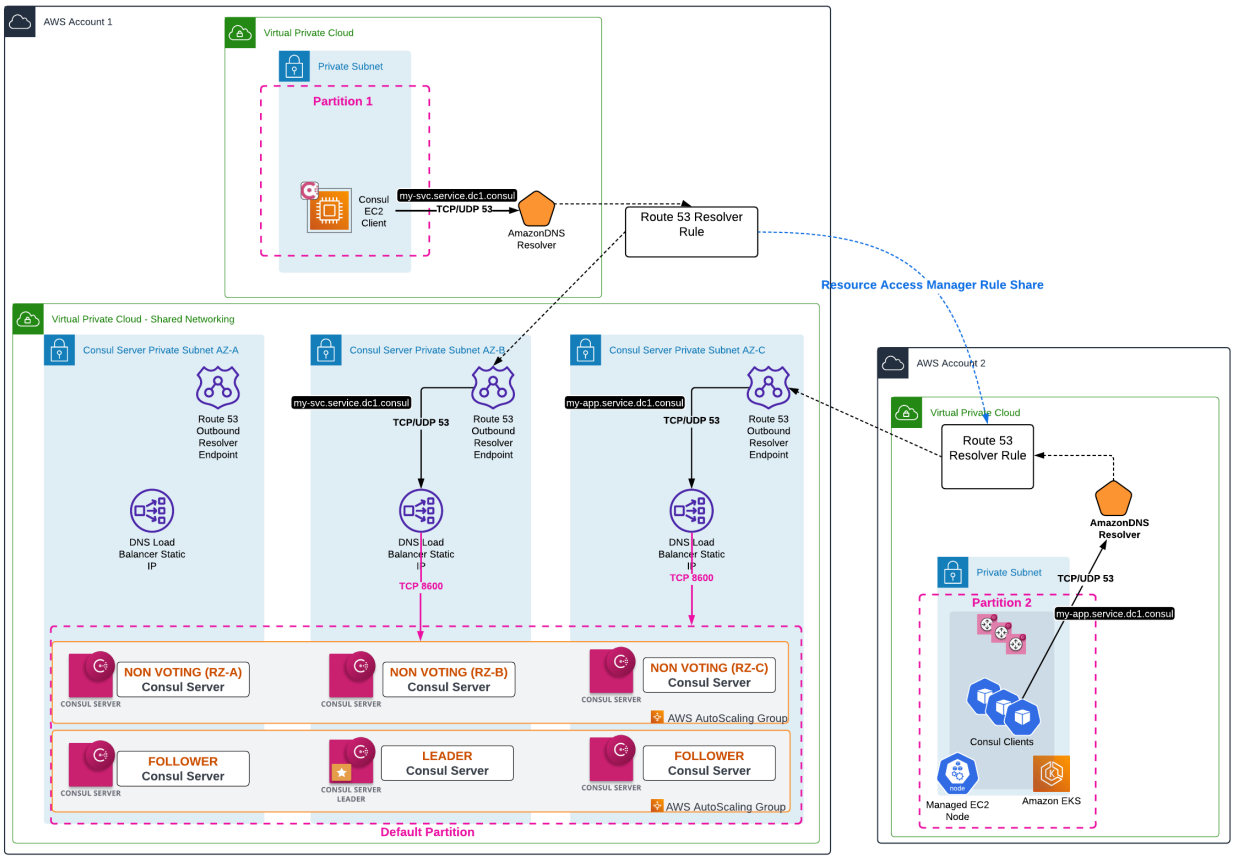


Figure 9: Consul DNS

Alternatively, if you are using another DNS service within your VPC (specified by setting the DHCP Option Set of the VPC), they should ensure that they are forwarding traffic appropriately within these systems. Examples of this could be, for example, utilizing Microsoft DNS, InfoBlox, or similar DNS software. These systems should be configured to forward requests related to Consul to the DNS service port of the cluster.

8.2.2 Security

Consul has various means of encrypting communication between nodes and components within the cluster. TLS Certificates are utilized to ensure secure Remote Procedure Calls (RPC) between clients and servers. We recommend securing all communication between agents, servers, and

other components via mutual TLS (mTLS) for RPC traffic. Finally, Gossip encryption is used to safeguard communication between Consul agents through the use of a shared symmetric key.

Consul requires certificates for various interfaces (HTTPS, gRPC, internal RPC) to be signed by one or more verifiable Certificate Authorities (CAs). There may be occasions that you would want to use a public CA for HTTPS and gRPC interfaces for public access to the Control Plane, and a private CA for internal RPC interfaces. However, we recommend that this Certificate Authority (CA) be a dedicated private CA for all interfaces. This is to ensure that only dedicated systems can communicate with Consul, and the Consul Control Plane remains within your network, limiting access external from the Data Center it runs in unless utilizing Consul Peering. Multiple CAs can be introduced at a future time if needed, such as for public access as previously mentioned, or for migration from one CA to another for agents.

Note

The Certificate Authority (CA) configured on the internal RPC interface (either explicitly by tls.internal_rpc or implicitly by tls.defaults) should be a private CA, not a public one. We recommend using a dedicated CA which should not be used with any other systems. Any certificate signed by the CA will be allowed to communicate with the cluster.

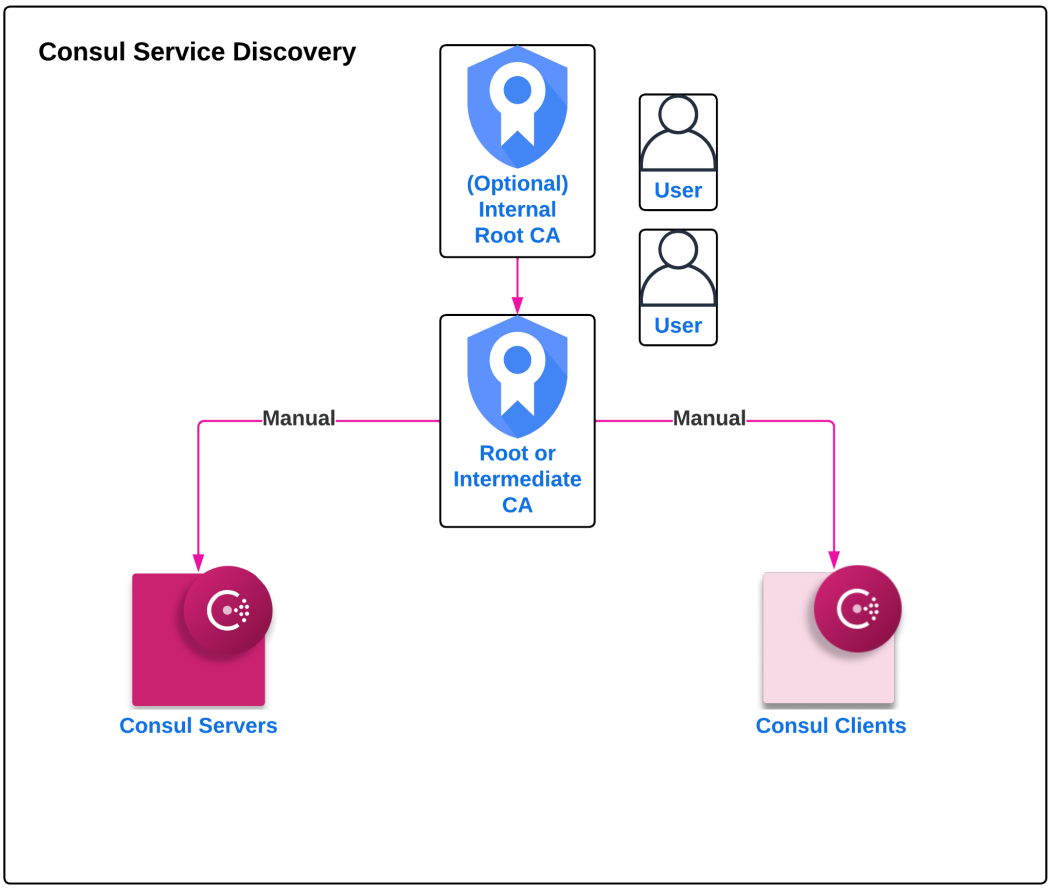


Figure 10: consul service discovery CA

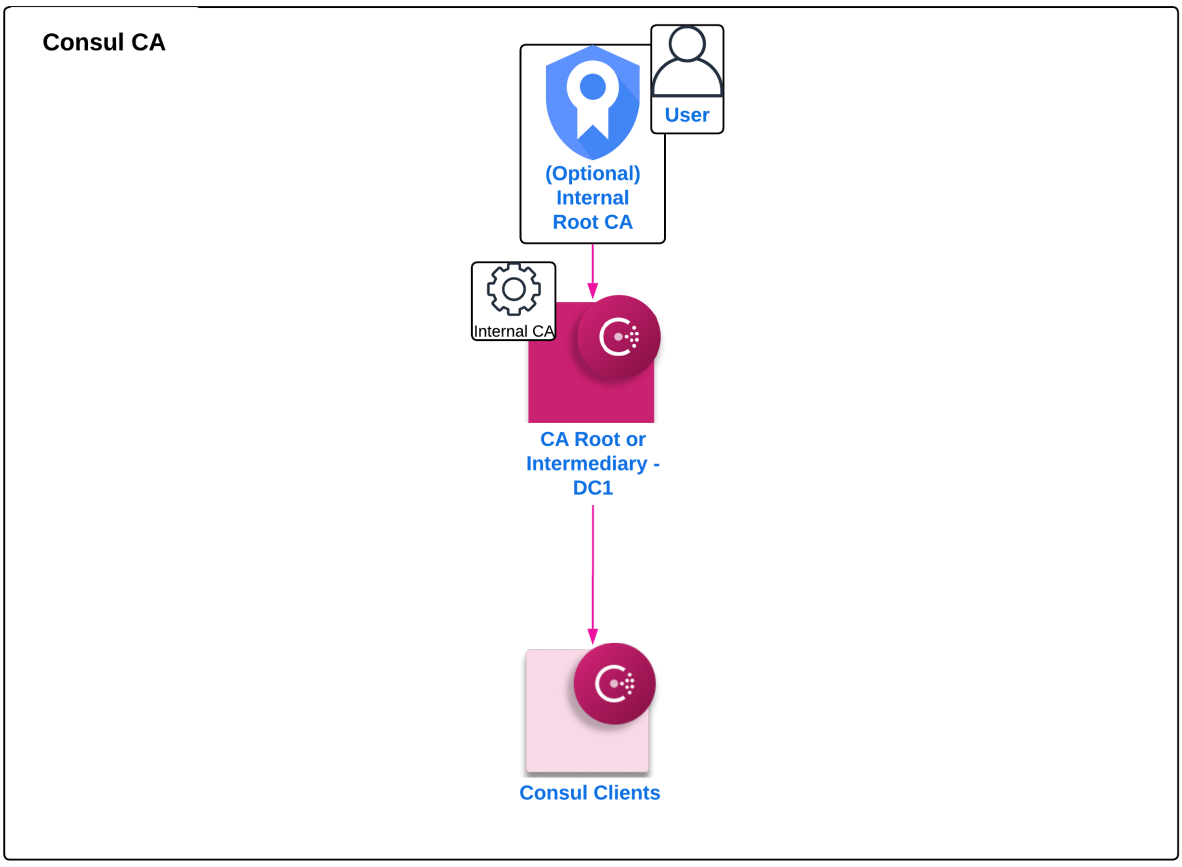


Figure 11: consul ca

If you do not currently have access to a Vault cluster, ensure that you leverage your organization's PKI process to provision certificates for the Servers and clients (note in this case, the certificate for the Agents will also be utilized for the Gateway components). Currently Consul requires that root certificates are valid SPIFFE SVID Signing Certificates, and that the URI encoded in the SAN is the cluster identifier created at time of bootstrap with the ".consul".

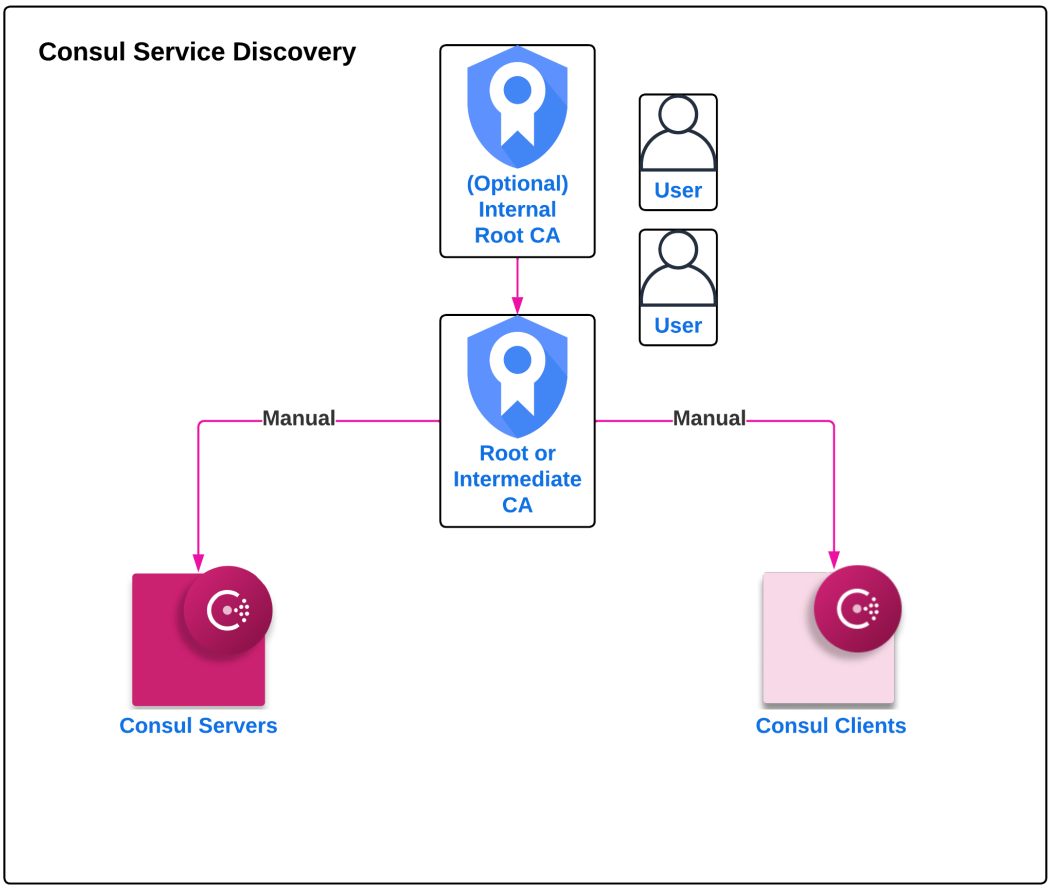


Figure 12: consul service discovery CA

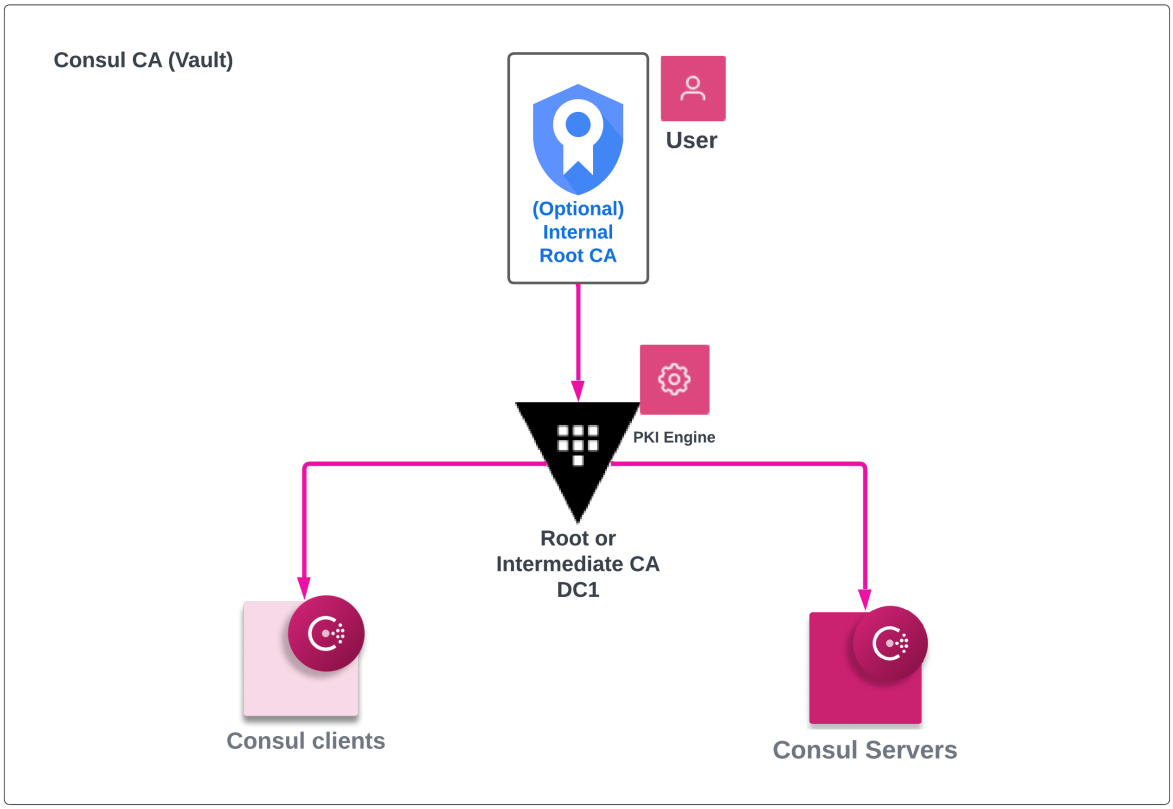


Figure 13: Vault as Consul CA

Note that for Control Plane, utilizing your internal CA is manageable for a Service Discovery and ServicerMesh use-case. For dataplane, we can use internal CA but must allow Consul to issue its own intermediate so it can generate its own certs under that chain.

As such, utilizing automated methods to issue certificates (via Vault or Consuls Internal CA acting as intermediary) is the recommended approach. For example, as the use case moves from Service Discovery to Service Mesh, having a Vault cluster in place, or utilizing Consuls built in CA allows automatically managed Leaf certificates for Mesh and Client certificates.

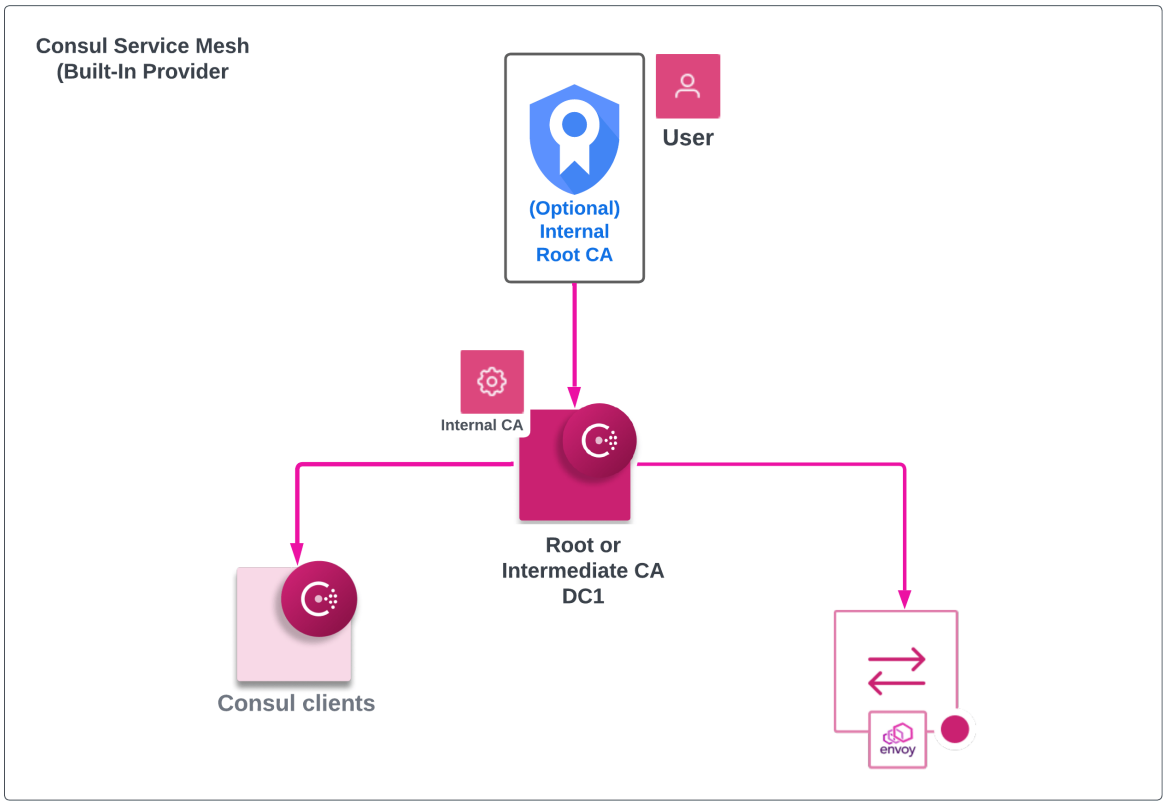


Figure 14: consul service Mesh CA

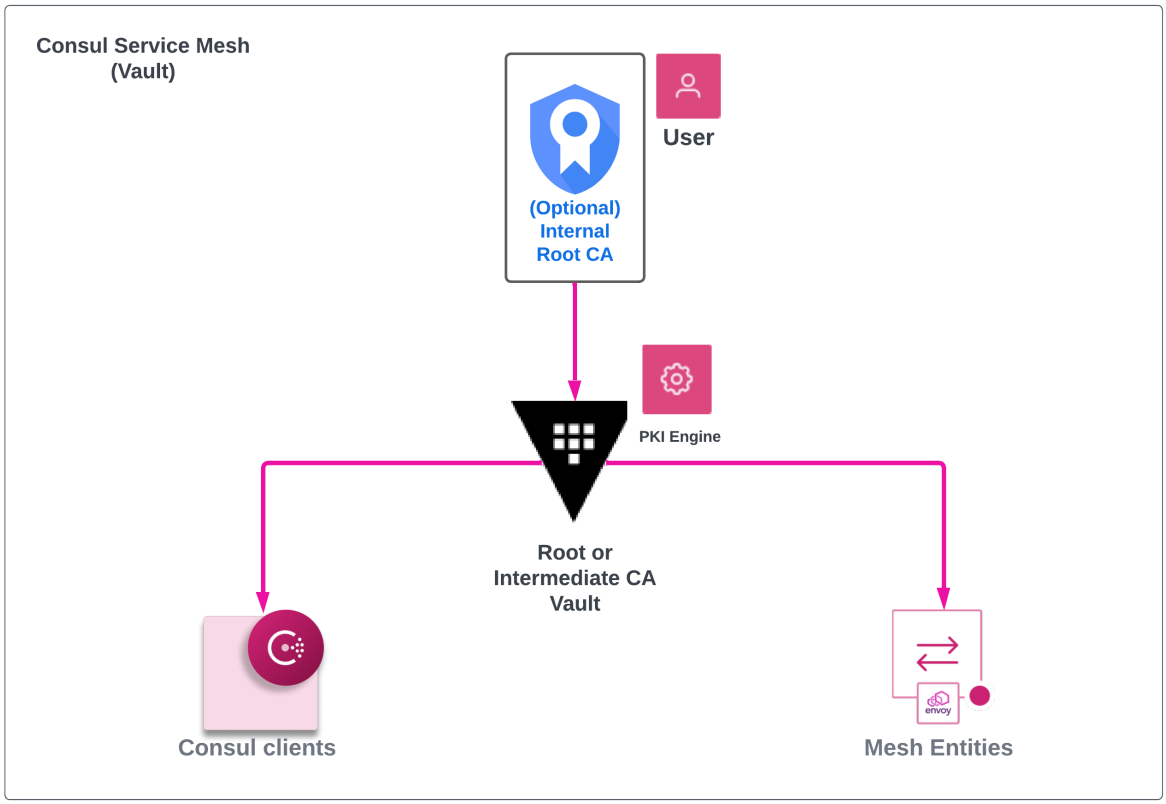


Figure 15: Vault as Consul CA

Only Consul controlplane have TLS configured with certificates containing the following SAN Fields.

Consul type	SAN format	Note
Client	N/A	All Consul clients should have unique names
Server	`server.<datacenter>.<domain>; <node_name>.server.<datacenter>.<domain>`	Required as a standard SAN name across all Consul Servers; Used for WAN Federation via Mesh Gateways

The addition of a Common SAN name is due to configuration settings within the Consul cluster. Specifically, `verify_server_hostname` performs hostname verification on all outgoing connections. This is a critical requirement that ensures that servers have this particular SAN in their certificate which in turn ensures that a client cannot modify its configuration and restart as a server.

You can set `verify_incoming` and `verify_outgoing` in interface specific stanzas, or as the default across all stanzas. You may want to do this to turn off incoming or outgoing verification at a protocol level.

Consul Config Settings	Details
`tls.defaults.verify_incoming`	Requires all incoming connections to use TLS. Valid for RPC and HTTPS API connectivity
`tls.defaults.verify_outgoing`	Requires all outgoing connections to use TLS. Applies to clients and servers
`tls.defaults.internal_rpc.verify_server_hostname`	Validates that a server has a SAN name present with `server.<datacenter>.<domain>`. Prevents clients from becoming server agents.

8.2.3 Secrets

Consul requires a number of secrets for various components, including its certificates as outlined above. Those secrets are as follows:

Type and purpose	Format
Gossip Key: Secures communication between agents	Base64 encoded 32 byte key
Bootstrap Token: Management token with unrestricted privileges	128-bit UUID
Agent Token: Used for internal agent operations.	128-bit UUID

Type and purpose	Format
DNS Token: Enables the agent to answer DNS queries about services and nodes.	128-bit UUID
Config File Service Registration Token: Registers services and checks from service and check definitions specified in configuration files or fragments passed to the agent using the `-hcl` flag.	128-bit UUID
Snapshot Token: Interacting with snapshot artifacts by snapshot agent	128-bit UUID
API GW Token: Grants permissions for API GW review binding	128-bit UUID
Terminating GW Token: Grants permissions to register into the catalog and to forward traffic out of the mesh	128-bit UUID
Mesh GW Token: Grants permissions to register into the catalog and routing to other services in the mesh or to other mesh gateways	128-bit UUID
ESM Token: Grants permission to ESM to monitor third-party or external services.	128-bit UUID

Each token must have an associated Accessor ID and policy associated with it. We recommend utilizing an existing Vault cluster for creation and storage of these tokens. This will allow ease of rotation, management, and synchronization for operations teams. Configuring Vault to create the Consul tokens is outside the scope of this Design Guide, however more information can be found here.

If you do not currently have access to a Vault cluster, and are following the Automated Implementation guide below these will be created for you during implementation and stored in AWS Secrets Manager. We recommend utilizing Secrets Manager on AWS for storing initial secrets of the Cluster until they can be updated as access control can be implemented with KMS Policy for decryption of the secret as well as access control via AWS IAM.

In addition to creating the ACL Tokens for each of the components, and its corresponding Accessor ID, you will need to also associate policies with each secret. This will be covered in the Operations Guide.

8.2.4 Monitoring

Monitoring Consul is crucial to ensure the health, performance, and reliability of your service discovery and configuration management infrastructure. Monitoring helps you identify issues, troubleshoot problems, and proactively manage your Consul cluster.

8.2.5 Metrics

Consul can be configured to send telemetry data to a remote monitoring system. This allows you to monitor the health of agents over time, spot trends, and plan for future needs. You will need a monitoring agent and console for this.

Consul supports the following telemetry agents:

- prometheus
- Circonus
- DataDog (via dogstatsd)
- StatsD (via statsd, statsite, telegraf, etc.)

If you are using StatsD, you will also need a compatible database and server, such as Grafana, Chronograf, or Prometheus.

Agent Telemetry can be enabled in the agent configuration file.

```
telemetry {
  dogstatsd_addr = "localhost:8125"
  disable_hostname = true
}
```

Consul on Kubernetes integrates with Prometheus and Grafana to provide metrics for Consul service mesh. The metrics available are:

- Mesh service metrics
- Mesh sidecar proxy metrics

- Consul agent metrics
- Ingress, terminating, and mesh gateway metrics

Prometheus annotations are used to instruct Prometheus to scrape metrics from Pods. Prometheus annotations only support scraping from one endpoint on a Pod, so Consul on Kubernetes supports metrics merging whereby service metrics and sidecar proxy metrics are merged into one endpoint. If there are no service metrics, it also supports just scraping the sidecar proxy metrics. Metrics for services in the mesh can be configured with the Helm values nested under connectInject.metrics.

```
connectInject:
  metrics:
    defaultEnabled: true # by default, this inherits from the value     global.
        metrics.enabled
    defaultEnableMerging: true
```

The Prometheus annotations specify which endpoint to scrape the metrics from. The annotations point to a listener on 0.0.0.0:20200 on the Envoy sidecar. You can configure the listener and the corresponding Prometheus annotations using the following Helm values. Alternatively, you can specify the consul.hashicorp.com/prometheus-scrape-port and consul.hashicorp.com/prometheus-scrape-path Consul annotations to override them on a per-Pod basis:

```
connectInject:
  metrics:
    defaultPrometheusScrapePort: 20200
    defaultPrometheusScrapePath: "/metrics"
```

Consul agent metrics

Metrics from the Consul server Pods can be scraped with Prometheus by setting the field global.metrics.enableAgentMetrics to true. Additionally, one can configure the metrics retention time on the agents by configuring the field global.metrics.agentMetricsRetentionTime which expects a duration and defaults to “1m”. This value must be greater than “0m” for the Consul servers to emit metrics at all. As the Prometheus deployment currently does not support scraping TLS endpoints, agent metrics are currently unsupported when TLS is enabled.

```
global:
  metrics:
    enabled: true
    enableAgentMetrics: true
    agentMetricsRetentionTime: "1m"
```

Gateway metrics

Metrics from the Consul ingress, terminating, and mesh gateways can be scraped with Prometheus by setting the field global.metrics.enableGatewayMetrics to true. The gateways emit standard Envoy proxy metrics. To ensure that the metrics are not exposed to the public internet, as mesh and ingress gateways can have public IPs, their metrics endpoints are exposed on the Pod IP of the respective gateway instance, rather than on all interfaces on 0.0.0.0.

```
global:
  metrics:
    enabled: true
    enableGatewayMetrics: true
```

Further, we need to authorize the Consul telemetry collector service to get metrics from other service proxies. We create service intentions that authorize proxies to push metrics to the collector.

8.2.6 Logs

Consul logs to standard output which can be redirected in your startup/init/systemd unit file or to any file you choose. That said, there is also a -syslog command line flag and a corresponding enable_syslog configuration file option that will enable Consul to log to syslog as well.

Additionally, you can specify the location of where you want the logs saved, the number of bytes that should be written to a log before it needs to be rotated, the duration a log should be written to before it needs to be rotated, and the maximum number of older log file archives to keep using the -log-file, -log-rotate-bytes, -log-rotate-duration and -log-rotate-max-files command line flags respectively.

You can also connect to any running Consul instance with consul monitor to view operational logging at any level from err to trace.

8.3 VM-based technical requirements

Table 4: Resource checklist for the AWS environment

Consul on VMs checklist

Infrastructure	
EC2 compute instance running Linux	Compute environment where Consul Enterprise agents run.
EC2 Auto Scaling launch template	A collection of parameters that support the instantiation of an EC2 instance by EC2 Auto Scaling Group rules.

Networking	
DNS name	A DNS hostname that resolves to the load balancer IP address in the local availability zone
VPC	Amazon Virtual Private Cloud where the service resides.
Regional subnets	Subnets are bound to one Availability Zone. We recommend a minimum of three subnets. Subnets are bound to one Availability Zone.
Security groups	Security Groups are applied at the Elastic Network Interfaces (ENI) of components inside of AWS. Used to control traffic inbound and outbound of the ENI
Network Load Balancer	The load balancer distributes incoming application traffic across multiple EC2 instances.
Amazon S3 object storage	Amazon object storage service.
IAM accounts	AWS Identity or identities that have assigned privileges to manage, access and affect the infrastructure.
Secrets manager	Used to manage initial Consul secrets

Networking	
KMS key	Generate an encryption key to apply encryption at rest for various AWS resources used to support Consul
Logging	S3 object storage to store logs and CloudWatch service to examine the logs.
AWS SSM	Allows remote SSH connectivity to private EC2 instances. Requires SSM agent to be installed and IAM permissions granted to EC2 instance profile.

8.3.1 Infrastructure

Compute

This current Installation Guide will deploy the infrastructure per the designs presented in the Consul on VM AWS Architecture section (or the Consul on Kubernetes AWS Architecture section).

HashiCorp Consul supports various Hardware Architectures and Operating Systems. You can find a complete list of supported installation platforms here. For purposes of this Installation Guide, while operating Consul Control Plane on VMs, we recommend utilizing a Linux distribution with an AMD64 architecture.

In addition, as mentioned above in the recommended deployment architecture, we typically recommend adopting Consul in two categories; an Initial deployment (to establish the technology and onboard towards adoption), followed by a scaled large deployment to support production operations. While we are referring to a single deployment within this document, it is recommended that there also be a test environment to try out new features and functionality while adopting and growing in maturity that is separate from the Production Consul cluster(s). These test clusters should remain in an Initial deployment size unless otherwise required to save on operational burden, costs, and resources. Please see the table below for recommended sizing guidance for the two types of deployment options.

Networking

This Installation Guide deploys a highly available architecture that spans 3 availability zones. A VPC configured with private subnets to deploy Consul control plane and gateway components on AWS EC2 VMs.

AWS VPC configured with security groups, routes to communicate with in VPC and to transit gateway for cross partition multi platform Service discovery and secure mesh communication requirements.

There will be two Load Balancers for AWS Network Load Balancers (NLB) in private subnets for internal mesh gateway traffic for cross VPC traffic. Also, deploying AWS Application Load Balancer in a public subnet for external Consul API gateway traffic.

Private network

We generally recommend that the Consul Control plane be deployed into Private Subnets. This reduces accessibility from outside of secure networks, and prevents accidental exposure and misconfiguration of the systems to be compromised from unauthorized access. Depending on your requirements, you may expose the Consul API with a Load Balancer in a Public Subnet to allow management of the Consul Cluster from the Internet. However, for purposes of this Installation Guide, we recommend that the Control Plane and access to it is located within Private Subnets.

We also recommend that Consul be deployed in a centralized service VPC, with access to other VPCs via networking pathways (Transit Gateway, VPC Peering, Direct Connect Gateway, VPNs, etc). This will allow the Control Plane to map various segments of services to different Admin Partitions, and will allow for ease of scalability with future Cluster Peering, connecting multiple Consul Clusters together.

Subnets

To ensure high availability and fault tolerance against data center issues within AWS, it is recommended to deploy the Consul Control Plane across multiple subnets. Each subnet corresponds to an Availability Zone (AZ), with each Region having at least three AZs. Therefore, we suggest deploying two Consul Servers in a minimum of three AZs, resulting in six servers within a Virtual Private Cloud (VPC).

Routing

To ensure optimal performance of Consul Service Discovery and, in a future stage, Consul Service Mesh, it is crucial to establish Layer 3 & Layer 4 network connectivity between all services that will utilize these features. Consul Agents register services from various locations within the network, while Consul Servers maintain registry information for service queries from clients. To facilitate this communication, Consul Agents must report node and service health to the Consul Cluster, with VM-based services requiring deployment on every computer node in the datacenter. This process is carried out via RPC (TCP 8300) and LAN Gossip (TCP/UDP 8301).

To ensure routability between the VPC where the Consul Control Plane is deployed and the networks in which services are deployed, we recommend leveraging multiple AWS Services such as VPC Peering or Transit Gateway.

Firewalls

As mentioned above in Routing, Consul Server agents should have L3 network connectivity to Consul Client agents running in various networks. In addition, Consul has a number of ports that are used between various components. You must ensure that these are allowed between Consul components at various points of traffic policy enforcement (for example, the Operating System, Security Group, Network Access Control List, and Firewall Appliance levels):

Name	Port/Protocol	Source	Destination	Description
RPC	8300/TCP	All agents (client & server)	Server agents	Used by servers to handle incoming requests from other agents.
Serf LAN	8301/TCP & UDP	All agents (client & server)	All agents (client & server)	Used to handle gossip in the LAN. Required by all agents.

Name	Port/Protocol	Source	Destination	Description
Serf WAN	8302/TCP & UDP	Server agents	Server agents	Used by server agents to gossip over the WAN to other server agents. Only used in multi-cluster environments.
HTTP/HTTPS	8500 & 8501/TCP	Localhost of client or server agent	Localhost of client or server agent	Used by clients to talk to the HTTP API. HTTPS is disabled by default.
DNS	8600/TCP & UDP	Localhost of client or server agent	Localhost of client or server agent	Used to resolve DNS queries.
gRPC (Optional)	8502/8503/TCP	Envoy Proxy	Client agent or server agent that manages the sidecar proxies service registration	Used to expose the xDS API to Envoy proxies. Disabled by default.; 8502 - Envoy to client xDS; 8503 - Dataplane to Consul Server and Peering Replication

Name	Port/Protocol	Source	Destination	Description
Sidecar Proxy	21000-21555/TCP	All agents (client & server)	Client agent or server agent that manages the sidecar proxies service registration	Port range used for automatically assigned sidecar service registrations.

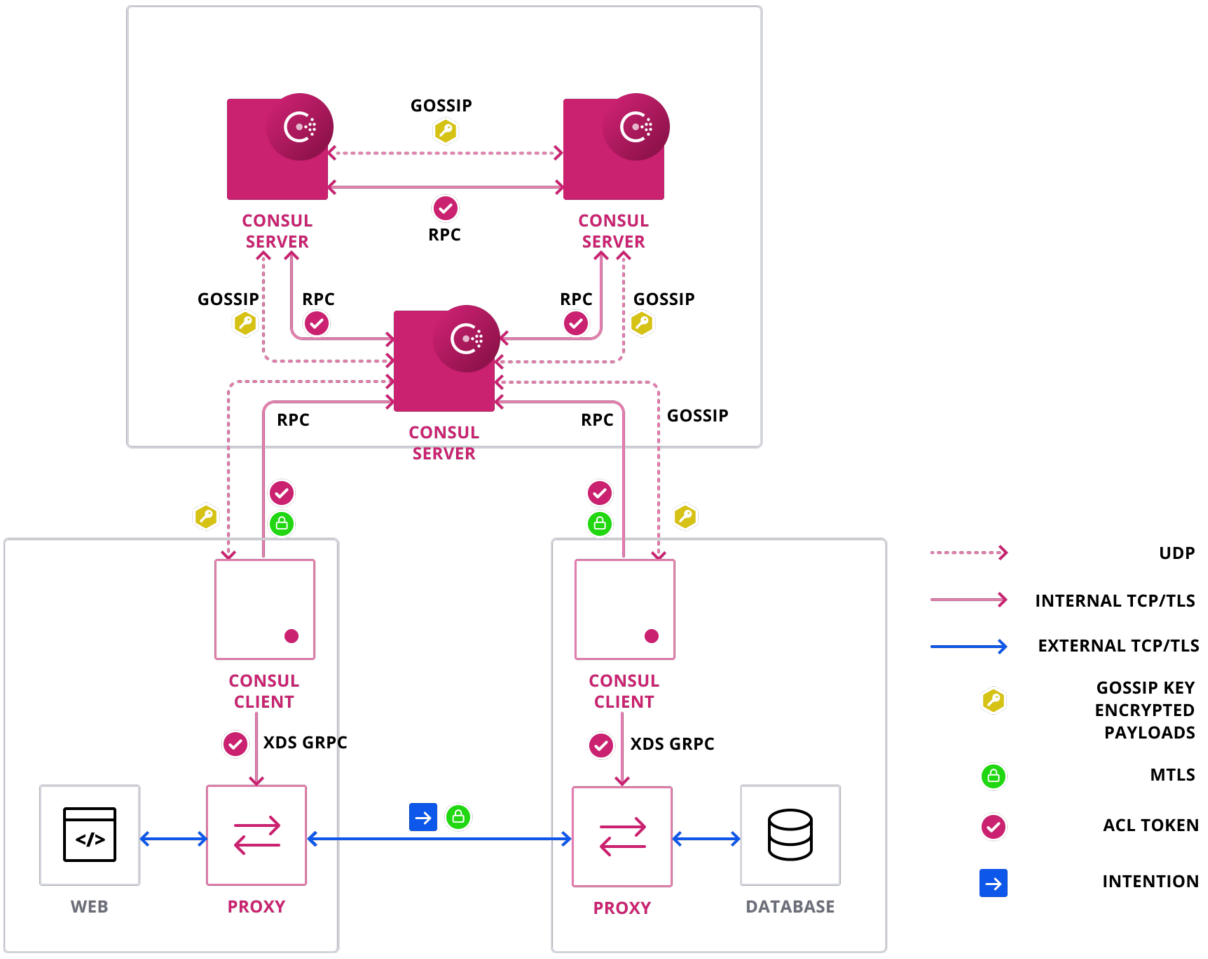


Figure 16: Consul Clients

Load balancing

There will be two Load Balancers in front of Consul and its components. AWS Network Load Balancers (NLB) in a private subnet for Server and internal mesh gateway traffic for intra and inter-VPC traffic and AWS Application Load Balancer in a public subnet for external Consul API gateway traffic.

Storage

Backup requirements It is important to take regular backups (what we will notate as snapshots) of your Consul cluster to maintain the registry of services. Consul Snapshot Agent runs within your environment as a service or scheduled to operate a highly available process that automatically takes snapshots, rotates backups, and sends the files to Amazon S3. We recommend provisioning an S3 bucket to use as part of this design. The snapshots are atomic and point-in-time, and include common items maintained by the Consul servers including (but not limited to):

- Key/Value Store Entries
- Service Catalog Registrations
- Prepared Queries
- Sessions
- Access Control Lists (ACLs)
- Namespaces

We recommend running the snapshot agent as a service (for example, with systemd) and utilize it as a long running process based on the RPO of your organization.

Disaster recovery

Consul Enterprise allows Disaster Recovery of multiple servers as well as cluster recovery. Redundancy Zones, Automated Backups, and Cloud Service Provider backup methodologies all aim at increasing the resiliency of the cluster in terms of an outage. As Consul leverages the Raft consensus protocol, if a single failed server is recoverable, bringing it back online involves two options; bringing the failed server back online with the same IP address, or forcing the server to leave and bringing on a new server with a new IP address. Autopilot helps with this process by cleaning up dead servers (by default, a check is run every 200ms) such that new servers can be created via an immutable process with new IPs and Autopilot will remove the previously failed server for you.

In the event of a loss of quorum in the Consul cluster or a complete loss of the Consul cluster, the quickest path to remediation would be to add new servers and then proceed with manual recovery with peers.json and same is true for the Kubernetes Consul recovery with exception.

Depending on the quorum size and number of failed servers, the recovery process will vary. In the event of complete failure it is beneficial to have a backup process.

As such, utilizing backup mechanisms of AWS is recommended. At a minimum, the S3 bucket that snapshots are being stored into should be replicated to a different bucket. We recommend this replicated bucket to be in a different AWS region; while S3 is Highly Available within the region, this will also protect against a regional failure of the cluster.

8.4 Kubernetes Based control plane technical requirements

Resource Checklist This resource checklist helps you prepare access to the resources required for the deployment. The checklist fits within a project plan and is used before the execution phase of your project.

For each item listed below, we briefly describe its relevance for the deployment. The technical requirements section of this document provides precise details on each requirement and a comprehensive description.

Table 5: Resource checklist for the aws environment Consul on kubernetes checklist

Infrastructure	
AWS EKS	Managed Kubernetes where Consul Enterprise runs.
EKS managed node groups	Automate the provisioning and lifecycle management of the EC2 based node groups. Kubernetes manages the pod scheduling on the node group and autoscale based on the need.

Networking	
DNS name	A DNS hostname that resolves to the load balancer IP address in the local availability zone
VPC	Amazon Virtual Private Cloud where the service resides.
Regional subnets	Subnets are bound to one Availability Zone. We recommend a minimum of three subnets. Subnets are bound to one Availability Zone.
Security groups	Security Groups are applied at the Elastic Network Interfaces (ENI) of components inside of AWS. Used to control traffic inbound and outbound of the ENI
Network loadbalancer	The load balancer distributes incoming application traffic across multiple EC2 instances.
Amazon S3 object storage	Amazon object storage service.
IAM accounts	AWS Identity or identities that have assigned privileges to manage, access and affect the infrastructure.
Secrets manager	Used to manage initial Consul secrets
KMS key	Generate an encryption key to apply encryption at rest for various AWS resources used to support Consul
Logging	S3 object storage to store logs and CloudWatch service to examine the logs.
AWS SSM	Allows remote SSH connectivity to private EC2 instances. Requires SSM agent to be installed and IAM permissions granted to EC2 instance profile.

Networking	
Helm	Required to deploy Consul Enterprise on EKS.

8.4.1 Infrastructure

8.4.2 Compute

We recommend using 2 EKS managed node groups for Control Plane (server agents) and Data Plane (client agents,service and Consul dataplane). This can be expanded further for monitoring stack and as per the organization's requirements.

Every managed node is provisioned as part of an Amazon EC2 Auto Scaling group that's managed for you by Amazon EKS. Every resource including the instances and Auto Scaling groups runs within your AWS account. Each node group runs across multiple Availability Zones that we define as part of the Installation Guide.

- Consul Server agents are configured as part of consul_server managed node group and provided with specific input as per cluster compute guidance.
- Consul Dataplane or services will be provisioned on consul_workload managed node groups as per the specified labels.

Managed groups configuration requirements can be applied to terraform.tfvars variables to apply specific infrastructure requirements.

- Pod sizing: Consul K8s has two recommendations. First, is to taint the servers node or create a separate node group to be only used by server agents (Both leader and non-voting servers agents) and secondly, the CPU and memory recommendations can be used when you select the resources limits for the Consul pods. The disk recommendations can also be used when selecting the resources limits and configuring persistent volumes. You will need to set both limits and requests in the Helm chart. Below is an example snippet of Helm configuration for a Consul server in a large environment.

```
server:
  resources: |
    requests:
      memory: "32Gi"
      cpu: "4"
    limits:
      memory: "32Gi"
      cpu: "4"

  storage: 50Gi
```

HashiCorp recommends monitoring your production deployment to make data-driven informed decisions to scale your production server resource limits or vertically scale the VM deployments in case of depleting VM CPU and memory resources. This will prevent common OOM-killed errors for server agents as they are resource intensive.

- **labels**: labels are required to provision any new instances of Consul server agents or client agents or services in their respective node groups.
- **Taints**: Server agents require dedicated nodes as they are heavy lifting the management of thousands of possible clients and services.

8.4.3 Networking

Private network

We generally recommend that the Consul Control plane be deployed into Private Subnets. This reduces accessibility from outside of secure networks, and prevents accidental exposure and misconfiguration of the systems to be compromised from unauthorized access. Depending on your requirements, you may expose the Consul API with a Load Balancer in a Public Subnet to allow management of the Consul Cluster from the Internet. However, for purposes of this Installation Guide, we recommend that the Control Plane and access to it is located within Private Subnets.

We also recommend that Consul be deployed in a centralized service VPC, with access to other VPCs via networking pathways (Transit Gateway, VPC Peering, Direct Connect Gateway, VPNs, etc). This will allow the Control Plane to map various segments of services to different Admin Partitions, and will allow for ease of scalability with future Cluster Peering, connecting multiple Consul Clusters together.

Subnets

To ensure high availability and fault tolerance against data center issues within AWS, it is recommended to deploy the Consul Control Plane across multiple subnets. Each subnet corresponds to an Availability Zone (AZ), with each Region having at least three AZs. Therefore, we suggest deploying two Consul Servers in a minimum of three AZs, resulting in six servers within a Virtual Private Cloud (VPC).

Routing

To ensure optimal performance of Consul Service Discovery and, in a future stage, Consul Service Mesh, it is crucial to establish Layer 3 & Layer 4 network connectivity between all services that will utilize these features.

To ensure routability between the VPC where the Consul Control Plane is deployed and the networks in which services are deployed, we recommend leveraging multiple AWS Services such as VPC Peering or Transit Gateway.

Firewalls

As mentioned above in Routing, Consul Server agents should have L3 network connectivity to Consul Client agents running in various networks. In addition, Consul has a number of ports that are used between various components. You must ensure that these are allowed between Consul components at various points of traffic policy enforcement (for example, the Operating System, Security Group, Network Access Control List, and Firewall Appliance levels):

Name	Port/Protocol	Source	Destination	Description
RPC	8300/TCP	All agents (client & server)	Server agents	Used by servers to handle incoming requests from other agents.

Name	Port/Protocol	Source	Destination	Description
Serf LAN	8301/TCP & UDP	All agents (client & server)	All agents (client & server)	Used to handle gossip in the LAN. Required by all agents.
Serf WAN	8302/TCP & UDP	Server agents	Server agents	Used by server agents to gossip over the WAN to other server agents. Only used in multi-cluster environments.
HTTP/HTTPS	8500 & 8501/TCP	Localhost of client or server agent	Localhost of client or server agent	Used by clients to talk to the HTTP API. HTTPS is disabled by default.
DNS	8600/TCP & UDP	Localhost of client or server agent	Localhost of client or server agent	Used to resolve DNS queries.

Name	Port/Protocol	Source	Destination	Description
gRPC (Optional)	8502/8503/TCP	Envoy Proxy	Client agent or server agent that manages the sidecar proxies service registration	Used to expose the xDS API to Envoy proxies. Disabled by default.; 8502 - Envoy to client xDS; 8503 - Dataplane to Consul Server and Peering Replication
Sidecar Proxy	21000-21555/TCP	All agents (client & server)	Client agent or server agent that manages the sidecar proxies service registration	Port range used for automatically assigned sidecar service registrations.

Load balancing

Consul EKS deployments require IAM role for service accounts (IRSA) policy and role configuration for the AWS Load Balancer controller.

All services that require a Load balancer need to specify it in the Helm definition and AWS Load Balancer Controller manager that provisions the required ELB.

Installation Guide helps you provision the required service accounts, roles, and AWS Load Balancer Controller Manager.

- Helm configuration - load balancer
- ELB_IRSA
- AWS Load Balancer Controller

Storage

Storage requirements for the Consul server agents or client services will be managed by Amazon Elastic Block Store (Amazon EBS) Container Storage Interface (CSI) driver that allows Amazon EKS clusters to manage the lifecycle of Amazon EBS volumes for persistent volumes.

EBS volume creation must be tagged with required resource tags with Data Lifecycle policies to prevent from deleting the EBS volumes and snapshots.

The Amazon EBS CSI driver isn't installed when you first create a cluster. To use the driver, you must add it as an Amazon EKS add-on.For instructions on how to add it as an Amazon EKS add-on, see Managing the Amazon EBS CSI driver as an Amazon EKS add-on.

- EBS CSI driver

Disaster recovery

To recover a Consul on Kubernetes primary datacenter from a disaster or during a long term outage you will need:

- A recent backup of Consul's internal state store
- A current backup of the Consul secrets

Consul Enterprise enables you to run the snapshot agent within your environment as a service (Systemd) or scheduled through other means. Once running, the snapshot agent service operates as a highly available process that integrates with the snapshot API to automatically manage taking snapshots, rotating backups, and sending backup files offsite to Amazon S3 (or another S3-compatible endpoint).

Configure Consul snapshot agent

Terraform deployment configures the Consul snapshot agent definition values to meet the organizations Recovery Point Objective (RPO) requirements. Update the snapshot_intervalvalues in the TF auto.tfvars file to apply the changes. This will help you configure the S3 bucket of choice in a given region in terms of disaster recovery scenarios and configure a snapshot interval to meet the RPO requirement for the organization's Service Level Agreement (SLA).

```
{
  "snapshot_agent": {
    "token": "",
    "datacenter": "",
    "ca_file": "",
    "ca_path": "",
    "cert_file": "",
    "key_file": "",
    "tls_server_name": "",
    "log": {
    "level": "INFO",
    "enable_syslog": false,
    "syslog_facility": "LOCAL0"
  },
  "snapshot": {
    "interval": "1h",
    "retain": 30,
    "stale": false,
    "service": "consul-snapshot",
    "deregister_after": "72h",
    "lock_key": "consul-snapshot/lock",
    "max_failures": 3,
    "local_scratch_path": ""
  },
  "local_storage": {
    "path": "."
  },
  "aws_storage": {
    "access_key_id": "",
    "secret_access_key": "",
    "s3_region": "",
    "s3_bucket": "",
    "s3_key_prefix": "consul-snapshot",
    "s3_server_side_encryption": false,
    "s3_static_snapshot_name": ""
  }
}
```

Helm configuration - Consul snapshot agent

```
# [Enterprise Only] Values for setting up and running snapshot agents
# (https://consul.io/commands/snapshot/agent)
# within the Consul clusters. They are required to be co-located with Consul clients,
# so will inherit the clients' nodeSelector, tolerations and affinity.
server:
  snapshotAgent:
    caCert: null
    configSecret:
    secretKey: null
    secretName: null
    enabled: true
    interval: 1h
    resources:
      limits:
      cpu: 50m
      memory: 50Mi
    requests:
      cpu: 50m
      memory: 50 Mi
```

Shared services

Certificates

Consul EKS deployment can enable TLS for Consul servers, clients, and allowable consul-k8s components. Enabling TLS will generate a certificate authority, and server and client certificates.

HashiCorp recommends to provide certificates that have been signed by a single certificate authority (CA) and update them as part of Kubernetes or Vault secret to enable TLS for the cluster.

The Helm configuration below enables TLS on the Consul cluster. More information regarding these configuration parameters can be found on the consul-helm repository.

Enable TLS - helm configuration

```
tls:
  enabled: true
  enableAutoEncrypt: true
  serverAdditionalDNSSANs: []
  serverAdditionalIPSANs: []
    verify: true
    httpsOnly: true
  caCert:
    secretName: null
    secretKey: null
  caKey:
    secretName: null
    secretKey: null
```

Secrets

Consul Kubernetes (EKS) deployment requires all the above specified required secrets. If not provided, Helm deployment will generate the secrets and update the Kubernetes secrets. If you specify Vault-as-backend, or provide your own secrets, you need to update the helm values for the Consul deployment.

9 Deploying Consul using Terraform

HashiCorp offers a set of official Terraform modules to simplify the deployment of a Consul Enterprise environment that align with the requirements and standards outlined in this HashiCorp Validated Design (HVD).

> Tip
>
> HashiCorp Professional Services or one of our trusted partners can help accelerate your Consul Enterprise deployment with confidence. For more information, reach out to your HashiCorp account team.

9.1 Platform-specific guidance

Select the tab below for your cloud for further guidance.

AWS

HashiCorp provides an official HVD module to deploy Consul Enterprise on AWS EC2.

GCP

HashiCorp provides an official HVD module to deploy Consul Enterprise on GCP GCE.

Azure

HashiCorp provides an official HVD module to deploy Consul Enterprise on Azure VMs.

By using this module, you can deploy a complete, end-to-end Consul Enterprise environment in your cloud account using Terraform.

Note

The steps provided offer a detailed, step-by-step method for installing Consul using this Terraform module. This method is intended for testing the module's functionality and familiarizing yourself with the design. We strongly recommend adapting these steps into a more controlled deployment process, such as using HCP Terraform, Terraform Enterprise, or integrating them into your existing CI/CD pipeline.

Warning

NEVER store Terraform state in a version control system or any other unprotected location. Terraform state files contain sensitive information and must be secured. For more information, refer to the Terraform documentation.

While we have provided prescriptive best practices throughout this guide, we understand that each organization has unique requirements and constraints for infrastructure deployment. We have included different considerations for deploying Consul in your cloud environment within the context of this module. If the default variables don't fully meet your needs, the module offers additional capabilities you may want to explore.

9.2 Deployment sequence overview

1. Create the TLS certificate files.

2. Obtain the Consul Enterprise license file.
3. Download the Consul CLI.
4. Download the Terraform CLI.
5. Set up your prerequisite resources.
6. Configure your cloud credentials.
7. Initialize your Terraform workspace.
8. Enter the required variables into the module for deployment.
9. Create a Terraform plan.
10. Apply the Terraform plan.
11. Verify that the cluster has been successfully created and is accessible.

9.3 Preparation

9.3.1 Create the TLS certificate files

Generate a standard X.509 certificate to be installed on the Consul servers. Follow your organization's process for creating a certificate that corresponds to the DNS record users will use to access Consul.

You will need the following three files.

- The certificate: `consul-crt.pem`
- The certificate's private key: `consul-key.pem`
- The CA bundle file from the certificate authority: `ca.pem`

Keep these files accessible, as they will be required later in the installation process.

9.3.2 Obtain the Consul Enterprise license file

Obtain the Consul Enterprise license file from your HashiCorp account team. This file includes a unique license key for your environment and will typically be named something like `consul.hclic`.

Keep this file accessible, as it will be required later in the installation process.

9.3.3 Download and install the Consul CLI

> Note
>
> This guide has been tested with Consul 1.19.2+ent.

Install the Consul CLI for your platform. You can do it manually by downloading the appropiate binary for your operating system, or using your preferred package manager. You can find instructions for the different options in here.

9.3.4 Download and install the Terraform CLI

> Note
>
> This guide has been tested with Terraform 1.9.6+ent.

Install the Terraform CLI for your platform. You can do it manually by downloading the appropiate binary for your operating system, or using your preferred package manager. You can find instructions for the different options in here.

9.3.5 Configure your cloud credentials.

Once you have downloaded the module, navigate to the examples/default/ directory. Use this as the base working directory during the installation process.

AWS

Ensure the appropriate AWS credentials are configured and accessible to Terraform. Terraform can access credentials from the following sources.* **Credentials file**: Typically located at `$HOME/.aws/credentials` (or `%UserProfile%\.aws\credentials` on Windows). * **Environment variables**: - `AWS_ACCESS_KEY_ID` - `AWS_SECRET_ACCESS_KEY` - `AWS_SESSION_TOKEN` (if using an IAM role or temporary credentials) - `AWS_DEFAULT_REGION`For detailed instructions on setting up AWS credentials for Terraform, refer to the Terraform AWS provider documentation.Make sure the credentials you use have the necessary permissions to perform the required actions in Terraform.

GCP

Ensure the appropriate credentials are set and accessible to Terraform.If you have GCloud CLI configured, you can run the following command to make your Google identity available:`bash gcloud auth application-default login`You can also impersonate a service account if needed. For detailed documentation on how to configure Google Cloud credentials for Terraform, refer to the Terraform Google Provider documentation.

Azure

Ensure the correct Azure credentials are in place and accessible to Terraform by running `az login` with your assumed credentials.For complete details on how to configure Azure credentials for Terraform, see the HashiCorp Terraform Azure provider documentation.Ensure that the credentials you will be using have sufficient permissions to perform the necessary actions that Terraform will be performing.

9.4 Installation

9.4.1 Set up your prerequisite resources

This section outlines the necessary infrastructure resources required to successfully execute the Terraform modules. These prerequisites include both infrastructure components and the appropriate storage locations for certificates, credentials, and licenses.

For more details about the required input variables, refer to the specific module's documentation.

AWS

HashiCorp provides an official HVD module to deploy Consul Enterprise on AWS EC2.Before deployment, you need to set up the required infrastructure in AWS.1) A functional VPC with at least three public and three private subnets. 2) A Consul license stored in the AWS Secrets Manager. - Store the content of the license file in a secret, using a name such as `consul_license_text`. The ARN of the secret will be used as an input to the Terraform module. 3) A TLS private key and certificate, valid for the fully qualified domain name intended for Consul, stored in the AWS Secrets Manager. - Store the TLS certificate, private key, and CA bundle as secrets using names like `consul_agent_cert`, `consul_agent_key`, and `consul_ca_cert`. The ARNs of these secrets will be used as input variables for the Terraform module. 4) An S3 Bucket for snapshots. - Ensure the S3 bucket is not publicly accessible and is encrypted. 5) SSH key name, already registered in AWS, to use for instance access. 6) An gossip encryption key for Consul. - You can use the Consul CLI's `consul keygen command` to generate a gossip encryption key. 7) An initial management token in the UUID format. - You can generate an initial management token using the Linux/UNIX command `uuidgen | tr "[:upper:]""[:lower:]"`.

GCP

HashiCorp provides an official HVD module to deploy Consul Enterprise on GCP GCE.Before deployment, you need to set up the prerequisite infrastructure in GCP:1. A GCP Project with required permissions. 2. A VPC with a subnet in a region with 3+ zones. 3. TLS private key and certificate, valid for the fully qualified domain name (FQDN) used with Consul, uploaded in PEM format and base64 encoded as secrets to GCP Secrets Manager. 4. A Consul Gossip Encryption Key uploaded as a secret to GCP Secrets Manager. 5. A Consul license uploaded as a secret to GCP Secrets Manager. 6. A GCS bucket for backup snapshot storage. ::: {data-environment=tcolorbox data-environment-keyword="colback=HexColorFFF9E8,colframe=HexColorFBEABF,coltitle=HexColor9E4B00,title=Note"} If it is not possible to have GCP project owner permissions, at least you will need permissions to: * Create IAM service accounts for the project. * Create/Update compute resources. * Create/Update networking resources. * Create/Update secrets in Secrets Manager.

Note

If your network and subnetwork are in a different project than Compute instances, the Terraform module will need read permissions to get networking data from the other project.

Azure

HashiCorp provides an official HVD module to deploy Consul Enterprise on Azure VMs.Before deployment, you will need to deploy the prerequisite infrastructure in Azure.1) An Azure resource group. 2) A virtual network with a private subnet for the Consul instances. 3) An Azure Key Vault to store the prerequisite secret material. 4) A Consul license stored as a secret in the Azure Key Vault. - Store the base64-encoded contents of the Consul license file as a secret, naming it `consul-license`. 5) An gossip encryption key for Consul. - Generate a gossip encryption key using the `consul keygen` command in the Consul CLI. - Base64 encode the key and store it as a secret named `gossip-key`. 6) A TLS private key and certificate, valid for the fully qualified domain name (FQDN) used with Consul, uploaded as secrets to the Azure Key Vault. - Upload the TLS certificate, private key, and CA bundle required for Consul's FQDN to the Azure Key Vault, they also need to be base64 encoded and stored as secrets named `consul_agent_cert`, `consul_agent_key`, and `consul_ca_cert`. 7) A storage account with an object container for Consul snapshot storage. - Store the primary access key of the storage account as a secret, using the name `storage-account-key`.

:::

9.4.2 Initialize your Terraform workspace

Run `terraform init` to initialize your Terraform workspace. Review the output to confirm that all providers and modules have been successfully downloaded and that no errors are present before proceeding.

9.4.3 Configure variables for the deployment

Note

You can configure variables in the installation module's terraform.tfvars file only after all prerequisite resources are set up. Values from the prerequisites will need to be supplied to the Consul module.

AWS

Create a file named `terraform.tfvars` in the `examples/amazonlinux-internal-nlb-consul-primary/` directory. Refer to the `terraform.tfvars.example` file for explanations of each variable, then populate your own file with the relevant values for your environment.

GCP

Create a file named `terraform.tfvars` in the `examples/ubuntu-nlb-consul-primary/` directory. Refer to the `terraform.tfvars.example` file for explanations of each variable, then populate your own file with the relevant values for your environment.

Azure

Create a file named `terraform.tfvars` in the `examples/ubuntu-nlb-consul-primary/` directory. Refer to the `terraform.tfvars.example` file for explanations of each variable, then populate your own file with the relevant values for your environment.

9.4.4 Create and apply the Terraform plan

In the example directory, generate a Terraform plan using the following command:

```
terraform plan
```

Review the output to understand the changes that will be made, then apply the changes with:

```
terraform apply
```

When prompted, type `yes` to confirm and proceed with the changes.

9.4.5 Verify that the cluster has been successfully created and is accessible

AWS

Once the Terraform run completes successfully, you can find the IP addresses of your Consul servers through the AWS console or by using the following command, specifying the name of the created autoscaling group, such as `production-consul-0.0.1`.shell aws ec2 describe-instances --instance-ids \ $(aws autoscaling describe-auto-scaling-groups \ --auto-scaling-group-name production-consul-0.0.1 \ --query "AutoScalingGroups[].Instances[].InstanceId"\ --output text)\ --query "Reservations[].Instances[].[InstanceId, PrivateIpAddress, PublicIpAddress]"\ --output tableTo connect to an EC2 instance using SSH, replace the IP address with the IP of one of the instances returned from the previous command.shell ssh ec2-user@<IP-ADDRESS>To configure the necessary environment variables for Consul, set the following values within your SSH session on the EC2 instance.shell export CONSUL_CACERT=/etc/consul.d/tls/consul-ca.pem export CONSUL_HTTP_ADDR=127.0.0.1:8501 export CONSUL_HTTP_SSL=**true** export CONSUL_HTTP_TOKEN=<aaaaaaaa-bbbb-cccc-dddd-eeeeeeeeeeee>To check the status of the raft peers in your Consul cluster and confirm the setup, run the following command.shell consul operator raft list-peersWith the recommended configuration, this command will display the status of the 6 Consul nodes in the cluster.

GCP

Once you have successfully deployed your Terraform configuration, you can view the IP addresses of the load balancer's frontend either in the GCP console or by executing the following command:shell gcloud compute forwarding-rules list --filter="name~'consul-forwarding-rule'"By default, in the Terraform module, the load balancer is configured internally (`load_balancing_scheme = "INTERNAL"`). To access the Consul cluster, you'll need to use the internal IP address of the forwarding rule and connect to the VPC via VPN (using Cloud NAT) or through a bastion host. For external access, configure the Terraform module variable for the load balancer with `load_balancing_scheme = "EXTERNAL"` and use the public IP address of the forwarding rule.The Consul bootstrap token will be securely stored as a secret in GCP Secret Manager during deployment. You can retrieve it using the following command:shell gcloud secrets versions access latest --secret consul-consul-management-tokenYou could set the following environment variables to interact with the Consul cluster:shell export CONSUL_HTTP_ADDR="https://$(gcloud compute forwarding-rules list --filter="name~'consul-forwarding-rule'"--format="value(IPAddress)"):8501 "export CONSUL_HTTP_TOKEN="$(gcloud secrets versions access latest --secret consul-consul-management-token)"Also, if the CA certificate is not in your trusted store, you can access to the Consul CA certificate using the secret name used during the deployment:shell gcloud secrets versions access latest --secret consul-tls-ca-cert | base64 -d > /path/to/consul-ca.pemThe previous command

10 Multi-cloud and multi-runtime deployment

This section describes how Consul will be installed and configured in a multi-cloud, multi-runtime environment. The configuration process involves several stages, including setting up Consul servers in multiple run-times (Kubernetes, VMs, and Nomad).

It also describes how to configure mesh gateways, enable cluster peering, and integrate with DNS and API gateways by following best practices for collaborative cloud connectivity and reliability.

10.1 Consul preparation

10.1.1 Deploy infrastructure

- **Kubernetes Environments:** Provision AWS EKS and Azure AKS clusters. Follow our recommendations for sizing and architecture described in the Consul Installation Guidelines.
- **EC2 Instances:** Provision EC2 instances that will host Consul and Nomad servers.

10.1.2 Prerequisites

Walk through the following guides:

- Configure Consul as code
- Consul on Kubernetes Helm Chart
- Nomad integration with Consul

10.1.3 Consul networking configuration

- Enable and set up mesh gateways.
- Enable and set up cluster peering.
- Set up DNS.
- Set up the API gateway.

10.1.4 Consul multi-tenancy configuration

- Enable admin partitions.

- Enable sameness groups.
- Configure service failover.

10.2 Consul installation and deployment

10.2.1 Install Consul on Kubernetes

This document describes the deployment of Consul on both Amazon EKS and Azure AKS clusters. To learn more about the requirements for each cloud provider, please refer to the "Deploy federated multi-cloud Kubernetes clusters" tutorial.

We recommend using the official Consul Helm chart to install Consul on Kubernetes, especially for multi-cluster setups that require cross-partition or cross datacenter communication. It simplifies and automates the deployment of all necessary components and supports various use cases, providing consistency and scalability.

```
global:
  name: consul
  image: "hashicorp/consul-enterprise:1.18.3-ent"
  adminPartitions:
    enabled: true

  acls:
    manageSystemACLs: true
  enableConsulNamespaces: true
  enterpriseLicense:
    secretName: "consul-ent-license"
    secretKey: "key"
    enableLicenseAutoload: true
  peering:
    enabled: true
  tls:
    enabled: true
server:
 replicas: 5
 bootstrapExpect: 5
 exposeService:
    enabled: true
    type: LoadBalancer

syncCatalog:
 enabled: true

ui:
  enabled: true
  service:
    enabled: true
    type: LoadBalancer
```

The same Helm chart can be used to install Consul on both clusters, allowing you to specify the datacenter and global name per cluster during deployment. For example, in an EKS or AKS setup, values can be directly specified through the command line:

EKS

```
$ helm install consul hashicorp/consul --set global.name=consul-eks --set global.
    datacenter=aws-eks-dc1
```

AKS

```
$ helm install consul hashicorp/consul --set global.name=consul-aks --set global.
    datacenter=aks-dc-1
```

10.2.2 Deploy Consul on EC2

To deploy Consul on EC2 using Terraform, HashiCorp provides Terraform modules that simplify the process and ensure a deployment following best practices and recommended architecture.

> **Note**
>
> Please contact your HashiCorp account team for access to these modules:
> terraform-aws-consul-prerequisites: Assists in provisioning prerequisite resources required for a Consul Enterprise deployment on AWS EC2. Infrastructure components include items such as network load balancers, KMS encryption keys, log groups, Route 53 entries, S3 buckets, etc.
> terraform-aws-consul: Deploys Consul Enterprise on AWS. It creates resources like AWS Auto Scaling groups, launch templates, security groups, and Route 53 configurations.

Here is an example of how you might configure these modules in your Terraform setup script:

```
module "consul_prerequisites" {
  source  = "hashicorp/consul/aws//modules/prerequisites"
  version = "x.y.z"

  # Add your specific configurations here
}

module "consul" {
  source  = "hashicorp/consul/aws"
  version = "x.y.z"

  # Add your specific configurations here
}
```

10.2.3 Setup Consul and Nomad

A Nomad cluster is typically comprised of a number of servers and client agents. Nomad differs slightly from Consul in that it divides infrastructure into regions which are served by one Nomad server cluster, but can manage multiple datacenters or availability zones. For examples, a US region can include datacenters `us-east-1` and `us-west-2`.

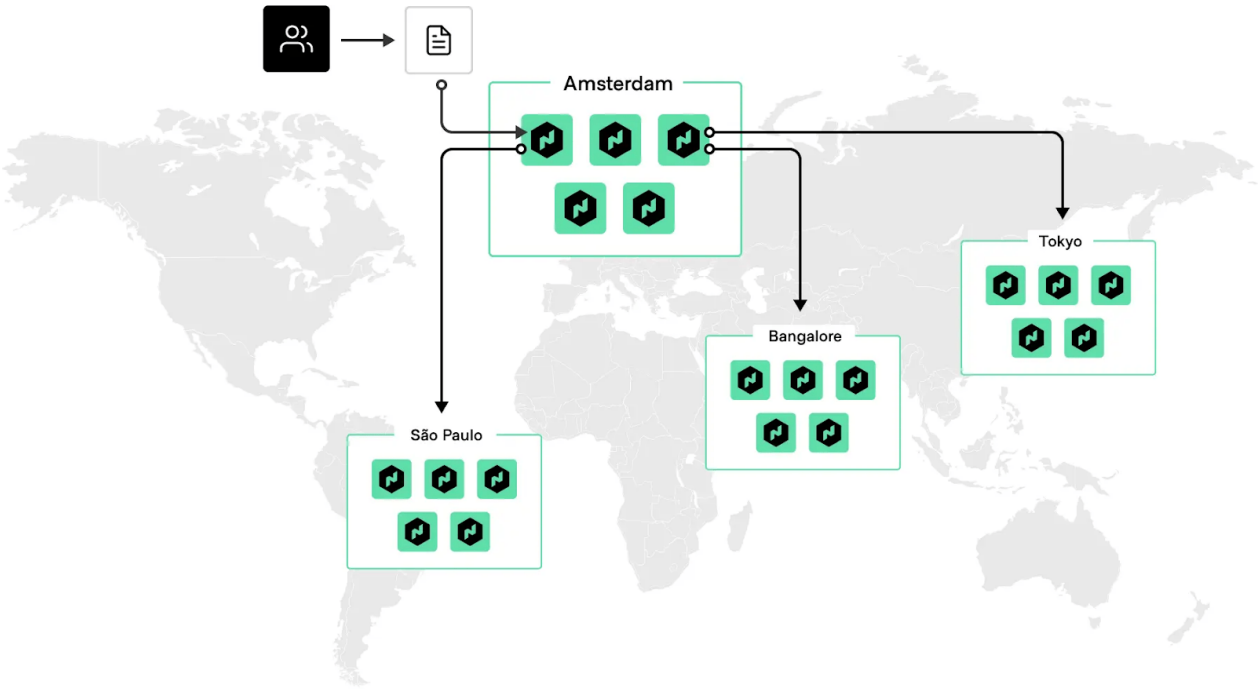


Figure 17: Consul and Nomad

A single Nomad cluster is recommended for applications deployed in the same region. Each cluster is expected to have three or five server, but no more than seven. This strikes a balance between availability in the case of failure and performance, as the Raft consensus protocol gets progressively slower as more servers are added.

Nomad Server Configuration

`/etc/nomad.d/server.hcl`

```
datacenter= "nomad-dc"
region = "us-west-1"
data_dir  = "/opt/nomad/data"
bind_addr = "0.0.0.0"
# Enable the server
server {
  enabled          = true
  bootstrap_expect = 5
}
consul {
  address = "127.0.0.1:8500"
  token = "CONSUL_TOKEN"
  client_auto_join = true
}

acl {
  enabled = true
}
```

It is recommended that you adjust certain Nomad settings (e.g. `raft_multiplier` and `retry_interval`) to optimize performance for cross-cloud communication latency.

Nomad Client Configuration

`/etc/nomad.d/client.hcl`

```
data_dir  = "/opt/nomad/data"
bind_addr = "0.0.0.0"
datacenter = "nomad-dc"

# Enable the client
client {
  enabled = true
  }
acl {
  enabled = true
}
consul {
  address = "127.0.0.1:8500"
  token = "CONSUL_TOKEN"
}
vault {
  enabled = true
  address = "http://active.vault.service.consul:8200"
}
```

10.3 Consul networking

10.3.1 Mesh gateways deployed over VMs

Deploying mesh gateways on VMs requires the Consul service mesh to be enabled. The flow of the control plane traffic through the gateway is implied by the presence of a mesh config entry with `PeerThroughMeshGateways = true`.

```
Kind = "mesh"
Peering {
  PeerThroughMeshGateways = true
}
```

10.3.2 Mesh gateways deployed on Kubernetes

In Kubernetes, mesh gateways can be enables in the Helm chart under the `global` section:

```
meshGateway:
  enabled: true
  replicas: 3
  service:
    type: NodePort
    nodePort: 30443
  resources:
    requests:
      memory: "256Mi" cpu: "100m"
    limits:
      memory: "512Mi"
      cpu: "500m"
  affinity: |
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector:
          matchLabels:
            app: {{ template "consul.name" . }}
            release: "{{ .Release.Name }}"
            component: mesh-gateway
          topologyKey: kubernetes.io/hostname
```

It is recommended that you:

- Assign at least three replicas for fault tolerance, and
- Add affinity so each mesh gateway pod is scheduled on a different node. See an example.

For all other supported parameters, refer to the Helm chart `meshGateway` documentation.

10.3.3 Mesh gateways in Nomad

The `gateway` block allows for configuration of Consul service mesh gateways. Nomad will automatically create the necessary gateway configuration entry, as well as inject an Envoy proxy task into the Nomad job to serve as the gateway. The gateway configuration is valid within the context of a `connect` block. Additional information about gateway configurations can be found in Consul's "Gateways Overview" documentation.

Example of a Nomad job definition including the mesh gateway:

```
job "countdash-mesh-one" {
  datacenters = ["aws-ec2"]

  group "mesh-gateway-aws-ec2" {
    network {
      mode = "bridge"

      # A mesh gateway will require a host_network configured on at least one
      # Nomad client that can establish cross-datacenter connections. Nomad will
      # automatically schedule the mesh gateway task on compatible Nomad clients.
      port "mesh_wan" {
        host_network = "public"
      }
    }

    service {
      name = "mesh-gateway"

      # The mesh gateway connect service should be configured to use a port from
      # the host_network capable of cross-datacenter connections.
      port = "mesh_wan"

      connect {
        gateway {
          mesh {
            # No configuration options in the mesh block.
          }

          # Consul gateway [envoy] proxy options.
          proxy {
            # The following options are automatically set by Nomad if not explicitly
            # configured with using bridge networking.
            #
            # envoy_gateway_no_default_bind = true
            # envoy_gateway_bind_addresses "lan" {
            #   address = "0.0.0.0"
            #   port    = <generated dynamic port>
            # }
            # envoy_gateway_bind_addresses "wan" {
            #   address = "0.0.0.0"
            #   port    = <configured service port>
            # }
            # Additional options are documented at
            # https://developer.hashicorp.com/nomad/docs/job-specification/gateway#
                proxy-parameters
          }
        }
      }
    }
  }
```



```
  group "dashboard" {
    network {
      mode = "bridge"

      port "http" {
        static = 9002
```

10.3.4 Setup DNS

Detailed instructions on configuring DNS for Kubernetes and VMs can be found in the Consul Installation Guide, under Post-installation configuration.

10.4 API Gateway Configuration

API gateways solve the following use cases:

- **Control access at the point of entry:** Set the protocols of external connection requests and secure inbound connections with TLS certificates from trusted providers, such as Verisign and Let's Encrypt.
- **Simplify traffic management:** Load balance requests across services and route traffic to the appropriate service by matching one or more criteria, such as hostname, path, header presence or value, and HTTP method.

10.4.1 API Gateway in Kubernetes

The API gateway is enabled in the Helm chart under the `connectInject` section. Once deployed, it will create a load balancer on each cloud provider automatically.

```
global:
  enabled: true
connectInject:
  apiGateway:
    managedGatewayClass:
      serviceType: LoadBalancer
```

To specify the gateway's behavior, entry point for north-south traffic and routing, it is necessary to define and deploy a gateway object as shown in the example:

```
apiVersion: gateway.networking.k8s.io/v1beta1
kind: Gateway
metadata:
  name: api-gateway
  namespace: consul
spec:
  gatewayClassName: consul
  listeners:
    - protocol: HTTP
      port: 80
      name: http
      allowedRoutes:
        namespaces:
          from: All
```

This configuration sets up an HTTP listener on port 80, allowing routes from all namespaces.

10.4.2 API gateway in Nomad

Provided is an example of Nomad's job definition, where an API gateway is deployed in its own Nomad namespace.

Start by creating a Nomad namespace:

```
$ nomad namespace apply -description "namespace for Consul API Gateways" ingress
```

Create a Consul ACL binding rule for the API gateway that assigns the `builtin/api-gateway` templated policy to Nomad workloads deployed in the Nomad namespace `ingress` that you just created:

```
$  consul acl binding-rule create -method 'nomad-workloads' -description 'Nomad API
    gateway' -bind-type 'templated-policy'-bind-name 'builtin/api-gateway' -bind-vars
     Name=${value.nomad_job_id}' -selector '"nomad_service" not in value and value.
    nomad_namespace==ingress'
```

The API gateway job needs mTLS certificates to communicate with Consul. Add certificates to the ingress namespace:

```
$ nomad var put -namespace ingress nomad/jobs/my-api-gateway/gateway/setup
    consul_cacert=@$CONSUL_CACERT consul_client_cert=@$CONSUL_CLIENT_CERT
    consul_client_key=@$CONSUL_CLIENT_KEY
```

10.4.3 Deploy the API gateway

Run the Nomad job. You can pass additional values to the command with the `-var` option.

```
$ nomad job run api-gateway.hcl
```

`api-gateway.hcl`

```
job "my-api-gateway" {

  namespace = var.namespace

  group "gateway" {

    network {
      mode = "bridge"
      port "http" {
        static = 8088
        to     = 8088
      }
    }

    consul {
      # If the Consul token needs to be for a specific Consul namespace, you will
      # need to set the namespace here

      # namespace = "foo"
    }

    task "setup" {
      driver = "docker"

      config {
        image = var.consul_image # image containing Consul
        command = "/bin/sh"
        args = [
          "-c",
         "consul connect envoy -gateway api -register -deregister-after-critical 10s
             -service ${NOMAD_JOB_NAME} -admin-bind 0.0.0.0:19000 -ignore-envoy-
             compatibility -bootstrap > ${NOMAD_ALLOC_DIR}/envoy_bootstrap.json"
        ]
      }

      lifecycle {
        hook = "prestart"
        sidecar = false
      }

      env {
        # these addresses need to match the specific Nomad node the allocation
        # is placed on, so it uses interpolation of the node attributes. The
        # CONSUL_HTTP_TOKEN variable will be set as a result of having template
        # blocks with Consul enabled.
        CONSUL_HTTP_ADDR = "https://${attr.unique.network.ip-address}:8501"
        CONSUL_GRPC_ADDR = "${attr.unique.network.ip-address}:8502" # xDS port (non-
            TLS)

        # these file paths are created by template blocks
        CONSUL_CLIENT_KEY  = "secrets/certs/consul_client_key.pem"
        CONSUL_CLIENT_CERT = "secrets/certs/consul_client.pem"
        CONSUL_CACERT      = "secrets/certs/consul_ca.pem"
      }

      template {
        destination = "secrets/certs/consul_ca.pem"
```

10.5 Setup cluster peering

It is recommended that you refer to the cluster peering section in the Administration Guide before proceeding with the next steps.

10.5.1 Establishing cluster peering connection diagram

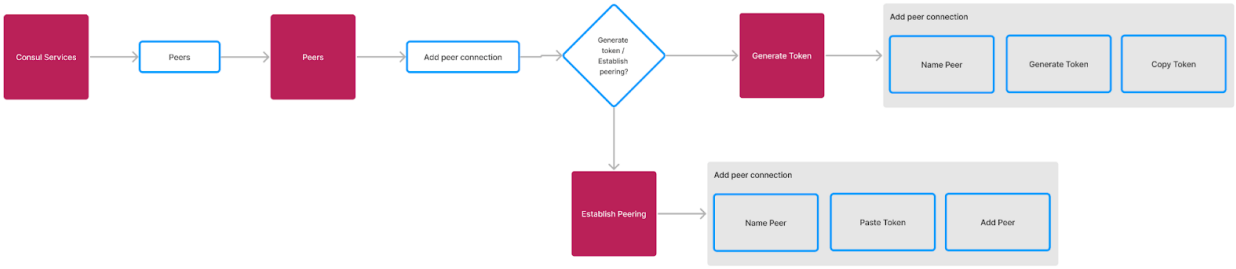


Figure 18: Cluster peering connection establishment

10.5.2 Enable cluster peering in Kubernetes

```
global:
  peering:
    enabled: true
```

10.5.3 Enable cluster peering for VMs

Cluster peering requires ACLs for authentication between clusters. Ensure that ACLs are enabled and properly configured:

```
acl {
  enabled = true
  default_policy = "deny"
  enable_token_persistence = true
}
```

10.5.4 Sameness groups

In Consul, sameness groups allow for service discovery and routing across multiple datacenters that are considered part of the same "group," which get treated equally for service routing purposes. When a failure occurs in one datacenter, Consul can seamlessly failover to another datacenter within the same sameness group.

Assuming the Counting application is deployed across Consul VMs and Nomad, and that the Dashboard application runs on Kubernetes (EKS and AKS), the configuration below presents an example on how to configure sameness groups:

`sameness.hcl` (VM)

```
service {
  name = "counting"
  id = "counting-vm"
  tags = ["sameness-group:counting"]
  port = 9001

  connect {
    sidecar_service {
      proxy {
        upstreams {
          destination_name = "dashboard"
          local_bind_port  = 8080
        }
      }
    }
  }

  check {
    name     = "HTTP health check"
    http     = "http://localhost:9001/health"
    interval = "10s"
  }
}
```

`counting-service.hcl` (Nomad)

```
job "counting" {
  group "counting-group" {
    task "counting-task" {
      driver = "docker"
      config {
        image = "your-counting-image"
      }

      service {
        name = "counting"
        tags = ["sameness-group:counting"]
        port = "9001"

        connect {
          sidecar_service {}
        }
      }

      check {
        type     = "http"
        path     = "/health"
        interval = "10s"
        timeout  = "2s"
      }
    }
  }
}
```

`sameness.yaml` (Kubernetes)

```
apiVersion: consul.hashicorp.com/v1alpha1
kind: SamenessGroup
metadata:
 name: dashboard
spec:
 defaultForFailover: true
 members:
   - peer: aws-ec2
```

10.5.5 Service failover

This configuration defines the default behavior for the `counting` service in Consul. It sets the service protocol to HTTP and assigns it the the “counting” sameness group for consistent behavior across environments. The `failover` section specifies that if the service in the primary datacenter

fails, traffic will automatically failover to the “counting” sameness group.

`servicefailovers.yaml` (Kubernetes)

```
aapiVersion: consul.hashicorp.com/v1alpha1
kind: ServiceResolver
metadata:
  name: counting
spec:
  defaultSubset: v1
  subsets:
      failover:
    v1:
      samenessGroup: "counting"
```

10.6 References

- “Helm Chart Reference”
- Nomad HVD
- “Nomad reference architecture”
- “API gateway for Kubernetes technical specifications”
- “Deploy a Consul API Gateway on Nomad”
- “Establish cluster peering connections on Kubernetes”
- “Establish cluster peering connections”
- “Create sameness groups on Kubernetes”

11 Post-installation configuration

After Consul Enterprise is running, complete the one-time configuration needed before handoff to operators and consumers. Ongoing IAM, namespace, partition, backup, restore, and upgrade management belongs in the Administration Guide.

11.1 Configure DNS forwarding

We recommend adopting a tiered approach to DNS forwarding with Consul Enterprise to get the best experience in each deployment variety. By prioritizing local agent resolution, the system can

achieve faster DNS lookups and aggregate requests through the client agent RPC connection, reducing network latency and providing the option to cache some portion of local DNS requests where sensible. This localized resolution not only enhances performance but also minimizes the dependency on external, centralized DNS servers, thereby reducing potential points of failure. As we move to systems without local client agents installed, because they haven't yet joined the cluster, or are still beginning to adopt Consul Enterprise, we can rely on external DNS resolution methods that are already in place within your environment and configure forwarding for the consul domain. For Kubernetes, leveraging CoreDNS further extends this flexibility, allowing for seamless service discovery across both containerized and traditional environments where you can expose routable NodePort, LoadBalancer service types or by registering your Kubernetes Ingress controller definitions using the catalog-sync daemon. This tiered approach ensures that DNS resolution is optimized for speed and locality when possible, while still maintaining the broad reach and compatibility required for diverse and distributed infrastructures.

11.1.1 Local agent resolution with DNS forwarding

We recommend starting with local agent resolution where you have Consul clients installed to achieve the best resolution locality, and optimal performance when considering TTLs and caching. By leveraging tools like `dnsmasq` or `systemd-resolved`, virtual machines equipped with client agents can resolve the `.consul` domain locally. This reduces the dependency and overall load on centralized DNS servers. For `dnsmasq`, a sample configuration would look like:

```
server=/.consul/127.0.0.1#8600
```

For `systemd-resolved`, in systemd version 246 and newer, the configuration would appear as:

```
[Resolve]
DNS=127.0.0.1:8600
DNSSEC=false
Domains=~consul
```

Many operating systems use a different hierarchy or toolchain for configuring local DNS forwarding or resolution, so it is important to understand the specifics of the Operating System version you are using. Many modern Linux distributions have standardized on systemd-resolved. More detailed configuration considerations can be found here.

A key advantage in this scenario is that your DNS queries will automatically have the context of your

client agent deployment, like the Partition your client is operating in when using admin partitions.

11.1.2 External resolution via DNS forwarding

For environments that require external DNS resolution because there are Consumers who haven't yet adopted Consul or don't have Consul agents deployed locally, using DNS Forwarding becomes essential. External forwarding can also be a useful way to find the Consul server addresses when initially joining the cluster because you can resolve the server IP addresses by referencing the `consul.service.consul` DNS query. Tools and platforms like Active Directory Conditional Forwarding, BIND, Route53, and Infoblox can be configured to forward requests for the `.consul` domain to the appropriate Consul servers. This approach ensures that even if a request originates outside the local environment, it's directed correctly to the Consul cluster for that Region. For instance, in BIND, a forward zone can be defined specifically for the `.consul` domain, ensuring that all such requests are forwarded to the Consul DNS interface. Each Region will have unique resolution targets for forwarding, so factoring that into your design is essential. The Installation Guide covers a method for forwarding within Amazon VPCs by leveraging Route53 outbound forwarding.

11.1.3 Kubernetes DNS forwarding with CoreDNS

In Kubernetes environments, CoreDNS is often the default DNS server. To integrate Consul's service discovery with Kubernetes, you can configure CoreDNS to forward requests for the `.consul` domain either to a ClusterIP DNS service within Kubernetes or an external load balancer exposing TCP and UDP DNS from a Consul agent. This ensures seamless service discovery across both Kubernetes and non-Kubernetes environments.

The following is a sample CoreDNS configuration to forward traffic for the consul domain to your Consul DNS endpoint. Add the following consul forwarding configuration to the Corefile section of your coredns ConfigMap located in the "kube-system" Kubernetes namespace.

```
. {
    forward . /etc/resolv.conf
    log
}

consul {
    forward . <CONSUL_DNS_TARGET>:<CONSUL_DNS_PORT>
    log
}
```

11.1.4 Forwarding destination for external resolution and kubernetes configurations

If you've been following the Installation Guide for deploying your Consul Enterprise servers into Amazon EC2, you should have a load balancer in front of your Consul server plane exposing DNS from the default partition on port 53 for both TCP and UDP. When deploying network load balancers within Amazon, the IP address of the listeners should not change once they are initially assigned. The forwarding destination for your external forwarding tier, or within CoreDNS, would need to be updated any time these IP addresses change due to re-deployment or other factors.

When forwarding DNS directly to the Consul server control plane, all DNS queries will be performed from the context of the default partition and namespace. You can include partition or namespace information inside your query when discovering services located outside of the default partition and namespace.

11.1.5 DNS ACL permissions

If you want to allow centralized DNS discovery across Consul namespaces and partitions within VM deployments you should adjust the anonymous token policy, or the default token policy of the DNS client agents to include cross partition and namespace read access. Currently this also grants read access when querying via the API.

```
partition_prefix "" {
    namespace_prefix "" {
        node_prefix "" {
            policy = "read"
        }
        service_prefix "" {
            policy = "read"
        }
    }
}
```

11.1.6 DNS performance considerations

To improve DNS scalability in larger environments, it is important to consider some key DNS configuration settings early on in your deployment.

Stale reads

By default, Consul enables stale reads and sets the `max_stale` value to 10 years. This allows Consul to continue serving DNS queries in the event of a long outage with no leader. The telemetry counter `consul.dns.stale_queries` can be used to track when agents serve DNS queries that are stale by more than 5 seconds.

```
dns_config {
    allow_stale = true
    max_stale = "87600h"
}
```

Negative response caching

Some DNS clients cache negative responses – for example, Consul returns a "not found" response because a service exists but there are no healthy endpoints. If you are using DNS for service discovery, cached negative responses may cause services to appear "down" for longer than they are actually unavailable.

One common example is when Windows defaults to caching negative responses for 15 minutes; DNS forwarders may also cache negative responses. To avoid this problem, check the negative response cache defaults for your client operating system and any DNS forwarder on the path between the client and Consul, and set the cache values appropriately. In many cases, an appropriate cache value may mean disabling negative response caching to get the best recovery time when a service becomes available again.

To reduce negative response caching, you can tune SOA responses and modify the negative TTL cache where you are servicing DNS requests.

```
dns_config {
    soa {
        min_ttl = 60
    }
}
```

TTL values

You can specify TTL values to enable caching for Consul DNS results. Higher TTL values reduce the number of lookups on the Consul servers and speed lookups for clients, at the cost of increasingly stale results. By default, all TTLs are zero which disables caching.

```
dns_config {
    service_ttl {
        "*" = "0s"
    }
    node_ttl = "0s"
}
```

You can specify a TTL that applies to all services or a more granular TTL for services that require more precision. The following example Consul server configuration enables external forwarding, or local agent configurations when local agent forwarding is configured. Consul will cache all node and service requests for 5 seconds, and cache results for the `web` service for 1 second.

```
dns_config {
    service_ttl {
        "*" = "5s"
        "web" = "1s"
    }
    node_ttl = "5s"
}
```

12 Identity integration — initial setup

This page covers the initial ACL bootstrap and authentication setup required before day-2 operations. Ongoing role, policy, namespace, and partition management belongs in the Administration Guide.

12.1 Configure initial ACL roles and policies

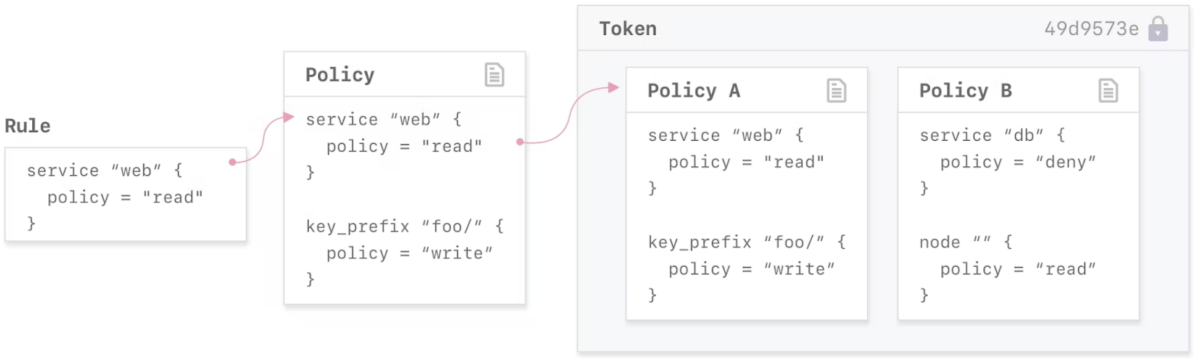


Figure 19: ACL Policies

You can use Consul ACL tokens to authenticate users, services, and Consul agents. When ACLs are enabled, entities requesting access to a resource must include a token that has been linked with a policy, service identity, or node identity that grants permission to the resource. The ACL system checks the token and grants or denies access to resources based on the associated permissions.

Authentication engines within Consul operate at the partition level. When you need roles or policies that grant access across more than one partition, you must define the auth method and policy within the "default" partition. All tokens generated within Consul are scoped to a specific partition and namespace.

12.1.1 Token precedence

The following hierarchy, applied in descending order, ensures structured access control in Consul, with each token type serving a distinct role in securing your environment.

1. **Tokens included in requests:** Tokens included in requests have the highest precedence. Consul evaluates these tokens first to determine request permissions.
2. **Default token:** In the absence of a request token, Consul considers the default token specified in the agent configuration. This provides a baseline level of access for requests made to the API that lack a specified token. This is commonly configured with read-only access to services to enable DNS service discovery on agents.

3. **Anonymous token:** The anonymous token applies when no other token is available, representing the permissions for unidentified requests.

Be mindful of token precedence when configuring your token so you can maintain a reliable and secure access control mechanism within your Consul setup.

12.1.2 Bootstrap the acl system

The first step for bootstrapping the ACL system is to enable ACLs on the Consul servers inside the agent configuration. With the architecture covered in this guide where the control plane is running on VM instances, the platform operator must configure the default ACL policy of "deny", which means the Consul Datacenter is configured to block access to all resources and functions by default unless access is explicitly granted via the token included in an API request, or included as part of the anonymous token rules. The `down_policy` of "extend-cache", means that the agents will ignore token TTLs during a control plane outage and continue operating based on the last cached state. If you have deployed your infrastructure following the Installation Guide, these configuration settings are already included in the default Terraform deployment configuration.

```
acl {
    enabled        = true
    default_policy = "deny"
    down_policy    = "extend-cache"
}
```

You can leverage Vault Enterprise Consul secrets engine to directly bootstrap the ACL system within Consul and protect the bootstrap token from being directly exposed to operators. In future HVD iterations, we will provide more prescriptive guidance on leveraging Consul Enterprise with Vault.

If you aren't using Vault Enterprise and are manually bootstrapping Consul's ACL system, after you enable ACLs in your server configuration you will manually bootstrap the ACL system using the Consul ACL bootstrap API.

The token returned, referenced in the `SecretID` field, is granted the highest level of access and shouldn't be used for extended periods of time. It is recommended that you only use this initial token to configure the first ACL policies and roles required for your infrastructure components and human operators by following the Installation Guide as well as the User Authentication section below. Once you have finished configuring your initial User Authentication roles and have validated their functionality, we recommend you discard the bootstrap token. The bootstrap token will only be

needed in the event that your human operators or GitOps pipelines lose access to Consul Enterprise due to issues with any of your authentication methods preventing them from functioning. In this scenario, where all human operators have lost access through Consul Authentication methods, you should generate a new management token by using the initial_management token configuration option on the Consul Server nodes.

```
$ curl --request PUT http://127.0.0.1:8500/v1/acl/bootstrap
{
    "AccessorID": "b5b1a918-50bc-fc46-dec2-d481359da4e3",
    "SecretID": "527347d3-9653-07dc-adc0-598b8f2b0f4d",
    "Description": "Bootstrap Token (Global Management)",
    "Policies": [
        {
            "ID": "00000000-0000-0000-0000-000000000001",
            "Name": "global-management"
        }
    ],
    "Local": false,
    "CreateTime": "2018-10-24T10:34:20.843397-04:00",
    "Hash": "oyrov6+GFLjo/KZAfqgxF/X4J/3LX0435DOBy9V22I0=",
    "CreateIndex": 12,
    "ModifyIndex": 12
}
```

12.1.3 Configure user authentication

One of the earliest pieces of configuration that will be established in your Consul Enterprise deployment is the User Authentication configuration. When establishing a new Consul Datacenter, the platform operators will leverage the Consul bootstrap token to carve out specific roles for different operational tasks. You will have elevated roles for tasks requiring a management token, as well as basic roles that will provide read-only visibility. We recommend leveraging the Consul Secrets Engine inside Vault as your identity broker for Consul Authentication where possible. Leveraging Vault allows you to login with any of Vault's supported authentication engines and map Vault roles to specific policies and roles within Consul. It also provides a central point to audit the mapping of short lived tokens to specific identities. If you are a customer without an established Vault deployment, or if you want a more native login experience for human operators within the UI and CLI, we recommend that you configure the single sign-on OIDC authentication method directly in Consul Enterprise. Detailed information about configuring the OIDC authentication method can be found here. We recommend that for human authentication workflows, defining all user authentication

methods within the "default" Partition, and then granting more granular access to specific Partitions using policy and role definitions targeting those Partitions is the best approach. This is because if you define policies and roles within a Partition directly, you cannot grant access across more than a single Partition which means you would need to logout and re-authenticate if you needed to administer across multiple Partitions. It is also important to include appropriate TTLs when configuring authentication methods that issue ACL tokens so that they will only be active for a short period of time if compromised.

The primary user authentication roles to configure for your initial Consul Enterprise Deployment are:

- **Cluster level operators:** For individuals who should be able to perform any function across the entire cluster in all Namespaces and Partitions we recommend issuing ACL tokens with the global management policy. Keep the number of global management ACL tokens limited to a small group of individuals.
- **Cross partition administrators:** This defines common functions that are used for the Service Discovery use case with administrative access across all Partitions, but doesn't cover every action that can be performed within Consul like the management token would. This role should be used for most Consul platform operators, and additional permissions could be extended here, if it is found that you are commonly needing to make changes using the global management ACL tokens.

```
partition_prefix "" {
  peering = "write"
  mesh = "write"
  acl = "write"
  namespace "default" {
    node_prefix "" {
      policy = "write"
    }
  }
  namespace_prefix "" {
    node_prefix "" {
      policy = "read"
    }
    key_prefix "" {
      policy = "write"
    }
    service_prefix "" {
      policy = "write"
    }
    session_prefix "" {
      policy = "write"
    }
  }
  agent_prefix "" {
    policy = "write"
  }
}

partition "default" {
  acl = "read"
}

query_prefix "" {
  policy = "write"
}

keyring = "read"
acl = "read"
operator = "read"
```

- **Cross partition read-only**: The policy "builtin/global-read-only" exists out of the box and allows Read-Only access across all Partitions and namespaces. This policy is useful for platform team members that need visibility into everything but that shouldn't be making changes directly.
- **Partition administrator**: Allows administrative functions within a specific Partition across all

Namespaces within that Partition. Also allows read access to prepared queries which are defined globally, and other global operator and ACL functions.

```
partition "<partition name>" {
    peering = "write"
    mesh = "write"
    acl = "write"
    namespace "default" {
        agent_prefix "" {
            policy = "write"
        }
        node_prefix "" {
            policy = "write"
        }
    }
    namespace_prefix "" {
        node_prefix "" {
            policy = "read"
        }
        key_prefix "" {
            policy = "write"
        }
        service_prefix "" {
            policy = "write"
        }
        session_prefix "" {
            policy = "write"
        }
    }
    agent_prefix "" {
        policy = "write"
    }
}

query_prefix "" {
    policy = "read"
}

keyring = "read"
acl = "read"
operator = "read"
```

- **Partition read-only**: Allows Read-Only access within a specific Partition, as well as all of the Namespaces within that Partition. Also allows read access to prepared queries which are defined globally, and other global operator and ACL functions.

```
partition "<partition name>" {
    peering = "read"
    mesh = "read"
    acl = "read"
    namespace_prefix "" {
        node_prefix "" {
            policy = "read"
        }
        key_prefix "" {
            policy = "read"
        }
        service_prefix "" {
            policy = "read"
        }
        session_prefix "" {
            policy = "read"
        }
    }
    agent_prefix "" {
        policy = "read"
    }
}

query_prefix "" {
    policy = "read"
}

keyring = "read"
acl = "read"
operator = "read"
```

12.1.4 Configure machine authentication

It is important to establish identity within your machine workloads early in your Consul Enterprise deployment. As you move towards a dynamic infrastructure deployment model, you can no longer depend on elements like the source IP address to represent specific workload security barriers. Instead, identity becomes the primary mechanism to secure and govern traffic patterns for your workloads across all Consul Enterprise use cases. Establishing identity is important because it ensures that services, upon startup and registration with the catalog, are actually who they claim to be. In service discovery, this means that only services with a validated identity should receive traffic based on consumer catalog queries. If you compare the Consul Service Catalog to a phone book, and Consul Enterprise to the telephone company, when you establish your telephone number

with the telephone company, they require you to present identification to establish your account and register an entry for you in the public phone records. If anyone could claim that they were someone else, there would be no value in the data within the phone book and people would no longer use it as a source of truth when trying to contact other people or businesses.

There are several machine authentication methods supported directly within Consul Enterprise based on your application deployment platform. When running workloads in Kubernetes, Amazon ECS, and Nomad, the secure introduction of Consul ACLs and tokens can be handled automatically for the consumer using the respective authentication methods or direct platform integration which greatly simplifies adoption. Alternatively, leveraging Vault Enterprise's Consul secrets engine as the identity broker is a great way to standardize on a central source of identity for all of your machine workloads. Manually distributing tokens to clients can be used as a last resort if you aren't able to leverage the native authentication methods, or Vault Enterprise.

For the highest level of security, individual service instances should have unique Consul ACL tokens. This improves auditability, and ensures you can reduce the blast radius in the event that an individual Consul Token is ever compromised. In cases where you are able to automate the distribution of new ACL tokens to consumers, it is also a good practice to issue ACL tokens with a shorter lived TTL to reduce the risk in the event of compromise.

A best practice is to never store Consul ACL Tokens directly in source code or configuration repositories. Retrieve ACL tokens dynamically at runtime where possible, or distribute static ACL tokens directly to machines using a secure configuration management approach.

Secure introduction of ACL tokens within organizations is a much larger topic which involves an organizational secrets management strategy. Refer to your Secrets Management platform for guidance on securely introducing sensitive values to your application workloads.

Authentication Methods Natively Supported in Consul Enterprise:

Types	Consul Version
kubernetes	1.5.0+
jwt	1.8.0+
OIDC	1.8.0+Enterprise
aws-iam	1.12.0+

12.1.5 Service identities

When distributing ACL tokens to individual applications, it is common to leverage service identities as a simple abstraction to tie a consumer to a common ACL policy. If your applications need permissions that aren't included as part of the service identity policy, you should define application-specific roles instead of leveraging service identities.

The following policy is generated for each service when a service identity is declared:

```
# Allow the service and its sidecar proxy to register into the catalog.
service "<service name>" {
    policy = "write"
}
# For service mesh use cases
service "<service name>-sidecar-proxy" {
    policy = "write"
}
# Allow for any potential upstreams to be resolved.
service_prefix "" {
    policy = "read"
}
node_prefix "" {
    policy = "read"
}
```

12.1.6 Agent tokens and node identities

The `acl.tokens.agent` is a special token that is used for an agent's internal operations. It isn't used directly for any user-initiated operations like the `acl.tokens.default`, though if the `acl.tokens.agent` isn't configured the `acl.tokens.default` will be used. The ACL agent token is used for the following operations by the agent:

- Updating the agent's node entry using the Catalog API, including updating its node metadata, tagged addresses, and network coordinates
- Performing anti-entropy syncing, in particular reading the node metadata and services registered with the catalog
- Reading and writing the special _rexec section of the KV store when executing consul exec commands

ACL agent tokens should be distributed to all of your server and client agent deployments. When distributing ACL agent tokens to your client and server agents, it is recommended to leverage Node

Identities scoped to the specific node name.

This policy is included when creating a node identity and is suitable for use as an ACL agent token:

```
node "<node name>" {
    policy = "write"
}
service_prefix "" {
    policy = "read"
}
```

12.1.7 AWS EC2 authentication

When deploying application workloads directly onto EC2 instances, we advise using the aws-iam authentication method. This method streamlines the secure introduction of Consul ACL Tokens by enabling you to build trust with your cloud platform and link your IAM instance profiles with policies and roles inside Consul Enterprise.

12.1.8 Configuration as code authentication

For a GitOps-driven configuration of Consul Enterprise, we suggest using workload identities in HCP Terraform and Terraform Enterprise, or a comparable JWT-based identity model linked to your specific CI/CD pipelines. Scope the workload identity with only the necessary permissions required to manage the configuration intended to be contained within that workspace or pipeline.

13 Observability setup

This page covers install-time telemetry and log wiring. Day-2 dashboard interpretation, alerting, rollout processes, and troubleshooting belong in the Administration Guide.

13.0.1 Monitoring

Monitoring Consul is crucial to ensure the health, performance, and reliability of your service discovery and configuration management infrastructure. Monitoring helps you identify issues, troubleshoot problems, and proactively manage your Consul cluster.

13.0.2 Metrics

Consul can be configured to send telemetry data to a remote monitoring system. This allows you to monitor the health of agents over time, spot trends, and plan for future needs. You will need a monitoring agent and console for this.

Consul supports the following telemetry agents:

- prometheus
- Circonus
- DataDog (via dogstatsd)
- StatsD (via statsd, statsite, telegraf, etc.)

If you are using StatsD, you will also need a compatible database and server, such as Grafana, Chronograf, or Prometheus.

Agent Telemetry can be enabled in the agent configuration file.

```
telemetry {
  dogstatsd_addr = "localhost:8125"
  disable_hostname = true
}
```

Consul on Kubernetes integrates with Prometheus and Grafana to provide metrics for Consul service mesh. The metrics available are:

- Mesh service metrics
- Mesh sidecar proxy metrics
- Consul agent metrics
- Ingress, terminating, and mesh gateway metrics

Prometheus annotations are used to instruct Prometheus to scrape metrics from Pods. Prometheus annotations only support scraping from one endpoint on a Pod, so Consul on Kubernetes supports metrics merging whereby service metrics and sidecar proxy metrics are merged into one endpoint. If there are no service metrics, it also supports just scraping the sidecar proxy metrics. Metrics for services in the mesh can be configured with the Helm values nested under connectInject.metrics.

```
connectInject:
  metrics:
    defaultEnabled: true # by default, this inherits from the value     global.
        metrics.enabled
    defaultEnableMerging: true
```

The Prometheus annotations specify which endpoint to scrape the metrics from. The annotations point to a listener on 0.0.0.0:20200 on the Envoy sidecar. You can configure the listener and the corresponding Prometheus annotations using the following Helm values. Alternatively, you can specify the consul.hashicorp.com/prometheus-scrape-port and consul.hashicorp.com/prometheus-scrape-path Consul annotations to override them on a per-Pod basis:

```
connectInject:
  metrics:
    defaultPrometheusScrapePort: 20200
    defaultPrometheusScrapePath: "/metrics"
```

Consul agent metrics

Metrics from the Consul server Pods can be scraped with Prometheus by setting the field global.metrics.enableAgentMetrics to true. Additionally, one can configure the metrics retention time on the agents by configuring the field global.metrics.agentMetricsRetentionTime which expects a duration and defaults to “1m”. This value must be greater than “0m” for the Consul servers to emit metrics at all. As the Prometheus deployment currently does not support scraping TLS endpoints, agent metrics are currently unsupported when TLS is enabled.

```
global:
  metrics:
    enabled: true
    enableAgentMetrics: true
    agentMetricsRetentionTime: "1m"
```

Gateway metrics

Metrics from the Consul ingress, terminating, and mesh gateways can be scraped with Prometheus by setting the field global.metrics.enableGatewayMetrics to true. The gateways emit standard Envoy proxy metrics. To ensure that the metrics are not exposed to the public internet, as mesh and ingress gateways can have public IPs, their metrics endpoints are exposed on the Pod IP of the respective gateway instance, rather than on all interfaces on 0.0.0.0.

```
global:
  metrics:
    enabled: true
    enableGatewayMetrics: true
```

Further, we need to authorize the Consul telemetry collector service to get metrics from other service proxies. We create service intentions that authorize proxies to push metrics to the collector.

13.0.3 Logs

Consul logs to standard output which can be redirected in your startup/init/systemd unit file or to any file you choose. That said, there is also a -syslog command line flag and a corresponding enable_syslog configuration file option that will enable Consul to log to syslog as well.

Additionally, you can specify the location of where you want the logs saved, the number of bytes that should be written to a log before it needs to be rotated, the duration a log should be written to before it needs to be rotated, and the maximum number of older log file archives to keep using the -log-file, -log-rotate-bytes, -log-rotate-duration and -log-rotate-max-files command line flags respectively.

You can also connect to any running Consul instance with consul monitor to view operational logging at any level from err to trace.

14 Backup and restore — initial setup

This page covers backup tooling configured at install time. Day-2 backup verification and restore procedures belong in the Administration Guide.

Storage

Backup requirements It is important to take regular backups (what we will notate as snapshots) of your Consul cluster to maintain the registry of services. Consul Snapshot Agent runs within your environment as a service or scheduled to operate a highly available process that automatically takes snapshots, rotates backups, and sends the files to Amazon S3. We recommend provisioning an S3 bucket to use as part of this design. The snapshots are atomic and point-in-time, and include common items maintained by the Consul servers including (but not limited to):

- Key/Value Store Entries
- Service Catalog Registrations
- Prepared Queries

- Sessions
- Access Control Lists (ACLs)
- Namespaces

We recommend running the snapshot agent as a service (for example, with systemd) and utilize it as a long running process based on the RPO of your organization.

Storage

Storage requirements for the Consul server agents or client services will be managed by Amazon Elastic Block Store (Amazon EBS) Container Storage Interface (CSI) driver that allows Amazon EKS clusters to manage the lifecycle of Amazon EBS volumes for persistent volumes.

EBS volume creation must be tagged with required resource tags with Data Lifecycle policies to prevent from deleting the EBS volumes and snapshots.

The Amazon EBS CSI driver isn't installed when you first create a cluster. To use the driver, you must add it as an Amazon EKS add-on.For instructions on how to add it as an Amazon EKS add-on, see Managing the Amazon EBS CSI driver as an Amazon EKS add-on.

- EBS CSI driver

15 Disaster recovery — initial setup

This page covers disaster-recovery design and initial recovery preparation at install time. Recurring failover tests and DR procedures belong in the Administration Guide.

Disaster recovery

Consul Enterprise allows Disaster Recovery of multiple servers as well as cluster recovery. Redundancy Zones, Automated Backups, and Cloud Service Provider backup methodologies all aim at increasing the resiliency of the cluster in terms of an outage. As Consul leverages the Raft consensus protocol, if a single failed server is recoverable, bringing it back online involves two options; bringing the failed server back online with the same IP address, or forcing the server to leave and bringing on a new server with a new IP address. Autopilot helps with this process by cleaning up dead servers (by default, a check is run every 200ms) such that new servers can be created via an immutable process with new IPs and Autopilot will remove the previously failed server for you.

In the event of a loss of quorum in the Consul cluster or a complete loss of the Consul cluster, the quickest path to remediation would be to add new servers and then proceed with manual recovery with peers.json and same is true for the Kubernetes Consul recovery with exception.

Depending on the quorum size and number of failed servers, the recovery process will vary. In the event of complete failure it is beneficial to have a backup process.

As such, utilizing backup mechanisms of AWS is recommended. At a minimum, the S3 bucket that snapshots are being stored into should be replicated to a different bucket. We recommend this replicated bucket to be in a different AWS region; while S3 is Highly Available within the region, this will also protect against a regional failure of the cluster.

Disaster recovery

To recover a Consul on Kubernetes primary datacenter from a disaster or during a long term outage you will need:

- A recent backup of Consul's internal state store
- A current backup of the Consul secrets

Consul Enterprise enables you to run the snapshot agent within your environment as a service (Systemd) or scheduled through other means. Once running, the snapshot agent service operates as a highly available process that integrates with the snapshot API to automatically manage taking snapshots, rotating backups, and sending backup files offsite to Amazon S3 (or another S3-compatible endpoint).

Configure Consul snapshot agent

Terraform deployment configures the Consul snapshot agent definition values to meet the organizations Recovery Point Objective (RPO) requirements. Update the snapshot_intervalvalues in the TF auto.tfvars file to apply the changes. This will help you configure the S3 bucket of choice in a given region in terms of disaster recovery scenarios and configure a snapshot interval to meet the RPO requirement for the organization's Service Level Agreement (SLA).

```
{
  "snapshot_agent": {
    "token": "",
    "datacenter": "",
    "ca_file": "",
    "ca_path": "",
    "cert_file": "",
    "key_file": "",
    "tls_server_name": "",
    "log": {
    "level": "INFO",
    "enable_syslog": false,
    "syslog_facility": "LOCAL0"
  },
  "snapshot": {
    "interval": "1h",
    "retain": 30,
    "stale": false,
    "service": "consul-snapshot",
    "deregister_after": "72h",
    "lock_key": "consul-snapshot/lock",
    "max_failures": 3,
    "local_scratch_path": ""
  },
  "local_storage": {
    "path": "."
  },
  "aws_storage": {
    "access_key_id": "",
    "secret_access_key": "",
    "s3_region": "",
    "s3_bucket": "",
    "s3_key_prefix": "consul-snapshot",
    "s3_server_side_encryption": false,
    "s3_static_snapshot_name": ""
  }
}
```

Helm configuration - Consul snapshot agent

```
# [Enterprise Only] Values for setting up and running snapshot agents
# (https://consul.io/commands/snapshot/agent)
# within the Consul clusters. They are required to be co-located with Consul clients,
# so will inherit the clients' nodeSelector, tolerations and affinity.
server:
  snapshotAgent:
    caCert: null
    configSecret:
    secretKey: null
    secretName: null
    enabled: true
    interval: 1h
    resources:
      limits:
      cpu: 50m
      memory: 50Mi
    requests:
      cpu: 50m
      memory: 50 Mi
```

Shared services

Certificates

Consul EKS deployment can enable TLS for Consul servers, clients, and allowable consul-k8s components. Enabling TLS will generate a certificate authority, and server and client certificates.

HashiCorp recommends to provide certificates that have been signed by a single certificate authority (CA) and update them as part of Kubernetes or Vault secret to enable TLS for the cluster.

The Helm configuration below enables TLS on the Consul cluster. More information regarding these configuration parameters can be found on the consul-helm repository.

Enable TLS - helm configuration

```
tls:
  enabled: true
  enableAutoEncrypt: true
  serverAdditionalDNSSANs: []
  serverAdditionalIPSANs: []
    verify: true
    httpsOnly: true
  caCert:
    secretName: null
    secretKey: null
  caKey:
    secretName: null
    secretKey: null
```

Secrets

Consul Kubernetes (EKS) deployment requires all the above specified required secrets. If not provided, Helm deployment will generate the secrets and update the Kubernetes secrets. If you specify Vault-as-backend, or provide your own secrets, you need to update the helm values for the Consul deployment.

16 Next steps

The Consul Enterprise deployment is now ready for operational handoff and consumer onboarding.

16.1 Operational handoff

Use the Consul Administration Guide to operate the platform, including identity and access management, observability, backups, restores, upgrades, support readiness, and multi-tenant governance.

16.2 Consumer onboarding

Use the Consul User Guide for service discovery, catalog usage, service mesh, deployment automation, and client-agent workflows.

16.3 Legacy summary folded into next steps

After installing HashiCorp Consul Enterprise, operational diligence is fundamental to ensuring a solid and reliable platform for your application developers to consume. This document has outlined numerous operational considerations, primarily focusing on embarking on your Consul Enterprise adoption journey with a spotlight on Service Discovery use cases. As you advance through your adoption journey, other HVD document iterations will cover additional considerations. Stay tuned for more HVD guidance covering these advanced topics in the future. Adhering to the guidelines laid out in this document will provide a robust foundation for managing and scaling your Consul deployment efficiently, thereby ensuring a resilient infrastructure aligned with your organization's operational standards.

Automating the deployment and management of Consul through CI/CD pipelines or GitOps workflows offers numerous advantages, and HashiCorp strongly advocates for this approach.

Key benefits include:

- **Increased Velocity:** Enables rapid updates and deployment of new services.
- **Standardization:** Allows operations and security teams to enforce consistent standards across environments.
- **Enhanced Security:** Improves the security posture by streamlining the patching and updating of systems, addressing vulnerabilities (CVEs) more effectively.
- **Cost Reduction:** Reduces the time and effort needed to update and maintain systems, resulting in significant cost savings.

By adopting automation, organizations can build a scalable, secure, and efficient Consul platform.

17 Appendix

17.1 Appendix 1: Admin partitions

Consul admin partitions give organizations the option to define tenancy boundaries for services using Consul. This can help organizations managing services across teams and business units. Teams can benefit from managing and customizing their own Consul environment, without impacting other teams, or other Consul environments.

Some organizations want to allow organizational or business units to deploy their own installations of Consul Enterprise on their own Kubernetes clusters. Centrally managing multiple Consul installations can be an operational challenge for organizations. Instead of giving teams their own clusters, the organization can consolidate these installations onto a shared multi-tenant server cluster. This cluster serves as the control plane for Consul clients in the tenant clusters, and ensures separation between tenants of the system. This deployment model provides teams the autonomy to configure Consul and application networking as they require. This increases the team's flexibility to manage application deployments, and eliminates the operational overhead associated with managing individual server clusters.

17.2 Appendix 2: Kubernetes catalog sync

To use catalog sync, you must enable it in the Helm chart. Catalog sync allows you to sync services between Consul and Kubernetes. The sync can be unidirectional in either direction or bidirectional. Refer to the Consul on Kubernetes section of the Installation Guide to learn more about the configuration options.

Services are synced from Kubernetes and non-Kubernetes to Consul service registry to make them discoverable, like any other service within the Consul datacenter. This service sync allows Kubernetes services to use Kubernetes' native service discovery capabilities to discover and connect to external services registered in Consul, and for external services to use Consul service discovery to discover and connect to Kubernetes services. Read more in the network connectivity section to learn more about related Kubernetes configuration. Services synced from Consul to Kubernetes are discoverable with the built-in Kubernetes DNS once a Consul stub domain is deployed. When bidirectional catalog sync is enabled, it behaves like the two unidirectional setups.

17.3 Appendix 3: Gossip protocol

Consul uses a gossip protocol to manage membership and broadcast messages to the cluster. The protocol, membership management, and message broadcasting is provided through the Serf library. The gossip protocol used by Serf is based on a modified version of the SWIM (Scalable Weakly-consistent Infection-style Process Group Membership) protocol.

Consul uses a LAN gossip pool and a WAN gossip pool to perform different functions. The pools are able to perform their functions by leveraging an embedded Serf library. The library is abstracted and

masked by Consul to simplify the user experience, but developers may find it useful to understand how the library is leveraged.

17.4 Appendix 4: Sample Terraform and helm configuration

17.4.1 Consul server

Terraform configuration

```
# Nodes tainted for running consul servers
consul_server = {
  name = "consul_server"

  instance_types = var.consul_server_node_type

  min_size   = 1
  max_size   = 5
  desired_size = var.consul_server_node_count

  // Nodes only for consul server agents; excluding other agents
  taints = {
    dedicated = {
      key   = "consul_agent_type"
      value = "server"
      effect = "NO_SCHEDULE"
    }
  }
  labels = {
    consul_agent_type = "server"
  }
}
```

Resource requirement (helm)

```
# Configure your Consul servers in this section.
server:
# Specify three servers that wait until all are healthy to bootstrap the Consul
    cluster.
replicas: 3
 # Specify the resources that servers request for placement. These values will serve
     a large environment.
resources:
  requests:
    memory: '32Gi'
    cpu: '4'
    disk: '50Gi'
  limits:
    memory: '32Gi'
    cpu: '4'
    disk: '50Gi'
```

17.4.2 Consul client

Terraform configuration

```
# Nodes to deploy services and consul client
consul_workload = {
  name = "consul_workload"

  instance_types = var.consul_workload_node_type

  min_size   = 0
  max_size   = 200
  desired_size = var.consul_workload_node_count
  labels = {
    app = "client - Data Plane"
  }
}
```

Resource requirement (helm)

```
# Configure Consul clients in this section
client:
# Specify the resources that clients request for deployment.
resources:
  requests:
    memory: '8Gi'
    cpu: '2'
    disk: '15Gi'
  limits:
    memory: '8Gi'
    cpu: '2'
    disk: '15Gi'
```

17.4.3 Helm configuration - load balancer

```
# ELB or Classic Load Balancer
---
global:
  name: consul
  datacenter: dc1
ui:
  enabled: true
  service:
    type: LoadBalancer
# NLB: Network Load Balancer
---
global:
  name: consul
  datacenter: dc1
ui:
  enabled: true
  service:
    type: LoadBalancer
    ingress:
     enabled: true
     ingressClassName: alb
     hosts:
      - host: consul-ui.test.consul.domain
     annotations: |
      'alb.ingress.kubernetes.io/certificate-arn': 'arn:aws:acm:us-east-2:01234
         xxxxxxx:certificate/f36b75c3-xxxx-40ca-xxx-3a2fad7f419d'
       'alb.ingress.kubernetes.io/listen-ports': '[{"HTTPS": 443}]'
       'alb.ingress.kubernetes.io/backend-protocol': 'HTTPS'
       'alb.ingress.kubernetes.io/healthcheck-path': '/v1/status/leader'
       'alb.ingress.kubernetes.io/group.name': 'envname-consul-server'

---
global:
  name: consul
  datacenter: dc1
ui:
  enabled: true
  service:
    type: LoadBalancer
    annotations: |
      'service.beta.kubernetes.io/aws-load-balancer-type': "external"
      'service.beta.kubernetes.io/aws-load-balancer-nlb-target-type': "instance"
```

17.4.4 ELB_IRSA - Terraform configuration

```
resource "kubernetes_service_account" "lb_controller" {
  metadata {
    name   = "aws-load-balancer-controller"
    namespace = "kube-system"
    labels = {
      "app.kubernetes.io/component" = "controller"
      "app.kubernetes.io/name"    = "aws-load-balancer-controller"
    }
    annotations = {
      "eks.amazonaws.com/role-arn" = module.lb_irsa.iam_role_arn
    }
  }
  depends_on = [
    module.eks
  ]
}
```

17.4.5 AWS load balancer controller - TF

```
resource "helm_release" "lb_controller" {
  name    = "aws-load-balancer-controller"
  repository = "https://aws.github.io/eks-charts"
  chart   = "aws-load-balancer-controller"
  namespace = "kube-system"

  set {
    name = "clusterName"
    value = module.eks.cluster_name
  }

  set {
    name = "serviceAccount.create"
    value = false
  }

  set {
    name = "serviceAccount.name"
    value = kubernetes_service_account.lb_controller.metadata[0].name
  }
}
```

17.4.6 EBS CSI driver

Terraform configuration for installing an EBS CSI driver.

```
data "aws_iam_policy" "ebs_csi_policy" {
  arn = "arn:aws:iam::aws:policy/service-role/AmazonEBSCSIDriverPolicy"
}

module "irsa-ebs-csi" {
  source =
   "terraform-aws-modules/iam/aws//modules/iam-assumable-role-with-oidc"
  version = "5.27.0"
  create_role          = true
  role_name           =
                      "AmazonEKSTFEBSCSIRole-${module.eks.cluster_name}"
  provider_url         = module.eks.oidc_provider
  role_policy_arns       = [data.aws_iam_policy.ebs_csi_policy.arn]
  oidc_fully_qualified_subjects =
             ["system:serviceaccount:kube-system:ebs-csi-controller-sa"]
}

resource "aws_eks_addon" "ebs-csi" {
  cluster_name       = module.eks.cluster_name
  addon_name        = "aws-ebs-csi-driver"
  addon_version      = "v1.24.0-eksbuild.1"
  service_account_role_arn = module.irsa-ebs-csi.iam_role_arn
  tags = {
    "eks_addon" = "ebs-csi"
    "terraform" = "true"
  }
  preserve = false
}
```

17.4.7 Consul server helm

```
global:

  name: consul
  image: "hashicorp/consul-enterprise:1.16.X-ent"
  datacenter: default
  adminPartitions:
    enabled: true
    name: "default"
  acls:
    manageSystemACLs: true
  enableConsulNamespaces: true
  enterpriseLicense:
    secretName: consul-enterprise-license
    secretKey: license
    enableLicenseAutoload: true
  peering:
    enabled: true
  tls:
    enabled: true

server:
 replicas: 3
 bootstrapExpect: 3
 exposeService:
    enabled: true
    type: LoadBalancer
 extraConfig: |
    {
      "log_level": "TRACE"
    }

syncCatalog:
 enabled: true
 k8sAllowNamespaces: ["*"]
 consulNamespaces:
   mirroringK8S: true

connectInject:
  enabled: true
  transparentProxy:
    defaultEnabled: false
  consulNamespaces:
    mirroringK8S: true
  k8sAllowNamespaces: ['*']
  k8sDenyNamespaces: []
  apiGateway:
    managedGatewayClass:
      serviceType: LoadBalancer

meshGateway:
  enabled: true
  replicas: 3
  service:
    enabled: true
```

17.4.8 Non-default partition K8s cluster - helm

```
global:
  name: consul
  datacenter: default
  enabled: false
  image: "hashicorp/consul-enterprise:1.16.X-ent"
  imageK8S: hashicorp/consul-k8s-control-plane:1.0.2
  imageConsulDataplane: "hashicorp/consul-dataplane:1.0.0"

  enableConsulNamespaces: true
  adminPartitions:
    enabled: true
    name: "prod-partition-1"
  peering:
    enabled: true

  tls:
    enabled: true
    caCert:
      secretName: consul-ca-cert
      secretKey: tls.crt
    caKey:
      secretName: consul-ca-key
      secretKey: tls.key

  acls:
    manageSystemACLs: true
    bootstrapToken:
      secretName: consul-bootstrap-acl-token
      secretKey: token

externalServers:
  enabled: true
  hosts: [ "a7d8d3f12cdfb4783af0357050e95416-secondryTest.us-east-1.elb.amazonaws.com
      " ] # External-IP (or DNS name) of the Expose Servers
  tlsServerName: server.default.consul   # <server>.<datacenter>.<dns>
  k8sAuthMethodHost: "CE8490A70630FBF25B9DsecondryTest.gr7.us-east-1.eks.amazonaws.
      com" # DNS name of EKS API of client1 - prod-parition-1
  httpsPort: 8501
  grpcPort: 8502
  useSystemRoots: false

server:
  enabled: false

connectInject:
  transparentProxy:
    defaultEnabled: true
  enabled: true
  default: true
  apiGateway:
    managedGatewayClass:
      serviceType: LoadBalancer

meshGateway:
  enabled: true
```

17.4.9 Agent telemetry

The Consul agent collects various runtime metrics about the performance of different libraries and subsystems. These metrics are aggregated on a ten second (10s) interval and are retained for one minute. An interval is the period of time between instances of data being collected and aggregated.

When telemetry is being streamed to an external metrics store, the interval is defined to be that store's flush interval.

External Store	Interval (Seconds)
dogstatsd	10s
Prometheus	60s
statsd	10s

Consul emits metrics under two major categories Consul health and server health:

Consul health:

- Transaction timing
- Leadership changes
- Autopilot
- Garbage collection

Server health:

- File descriptors
- CPU usage
- Network activity
- Disk activity
- Memory usage

17.4.10 Consul telemetry collector intention

Create a service-intentions configuration entry that allows all traffic to consul-telemetry-collector:

```
# consul-telemetry-collector.yaml
apiVersion: consul.hashicorp.com/v1alpha1
kind: ServiceIntentions
metadata:
  name: consul-telemetry-collector
spec:
  destination:
    name: consul-telemetry-collector
  sources:
  - action: allow
    name: '*'
```

Create configuration entry:

```
kubectl apply --namespace consul --filename consul-telemetry-collector.yaml
```

17.4.11 Consul server sizing - EC2 to Consul cluster size

Provider	Size	Instance Type	CPU	Memory	Disk Capacity	Disk IO
AWS	Small	m5.large	2	8	min: 100 GB (gp3)	min: 3000 IOPS
AWS	Medium	m5.xlarge	4	16	min: 100 GB (gp3)	min: 3000 IOPS
AWS	Large	m5.2xlarge	8	32	min: 200 GB (gp3)	min: 7500 IOPS
AWS	Extra Lage	m5.4xlarge	16	64	min: 200 GB (gp3)	min: 7500 IOPS

17.4.12 Agents

You can run the Consul binary to start Consul agents, which are daemons that implement Consul control plane functionality. You can start agents as Servers or clients.

terraform.auto.tfvars - Terraform deployment

```
friendly_name_prefix = "consul"

common_tags = {
  deployment = "consul"
  site       = "westeros"
}

route53_failover_record = {
  record_name = "consul"
}

secretsmanager_secrets = {
  license = {
    name = "consul-license"
    data = ""
  }
  ca_certificate_bundle = {
    name = "consul-ca-bundle"
    path = "./consul-agent-ca.pem"
  }
  cert_pem_secret = {
    name = "consul-public"
    path = "./consul-server-public.pem"
  }
  cert_pem_private_key_secret = {
    name = "consul-private"
    path = "./consul-server-private.pem"
  }
  consul_initial_management_token = {
    generate = true
  }
  consul_agent_token = {
    generate = true
  }
  consul_gossip_key = {
    generate = true
  }
  consul_snapshot_token = {
    generate = true
  }
  consul_ingress_gw_token = {
    generate = true
  }
  consul_terminating_gw_token = {
    generate = true
  }
  consul_mesh_gw_token = {
    generate = true
  }
}
```



```
snapshot_interval = "5 min"

s3_buckets = {
  snapshot = {
    bucket_name   = "consul-westeros-snapshots"
    force_destroy = true
```

18 Changelog

This page records notable changes to the Consul Installation Guide.

18.1 2026-08

- Restructured legacy Consul HVD content into the HVD 2.0 Consul Installation Guide as part of the Installation / Administration / User guide migration.

18.2 2026-01

- Update meta descriptions
- Updated credits and contributors

18.3 2025-06

- Use consistent guide names and descriptions

18.4 2025-05

- Removed document structure section

18.5 2025-01

- Add Consul Solution Design Guide updates for HVD Modules
- Updated documentation links

18.6 2024-02

- GA Release

19 Credits

Thank you to everyone who contributed to this validated design guidance. This cross-functional team, representing many departments within HashiCorp, authored, edited, reviewed, and iterated this content to provide prescription and recommendations for using HashiCorp software in enterprise scenarios.

List of key team members who contributed to the development of this guide. Hailing from various functional areas and representing diverse departments within HashiCorp, these experienced professionals have played pivotal roles in writing, editing, reviewing, and ensuring the completion of this material. Their collective efforts have been instrumental in shaping the final output.

- Abhishek Tiwari
- Adam Cavaliere
- Austin Workman
- Blake Covarrubias
- Çetin Ardal
- Chris Dunlap
- Cooper Melgreen
- David Cañadillas
- David Canadillas
- Gerald Dykeman
- Hari Sankaran
- Jan Repnak
- Josh Wolfer
- Kalen Arndt
- Kulpreet Singh
- Luke Kysow
- Mara Hammond
- Mark Lewis
- Pablo Diaz
- Prabhjit Singh
- Sean Doyle
- Srini Nagaraju
- Tu Nguyen